

CSCI 598H

Applied Computer Vision for Engineering Systems

Lecture 6-2: Transformer-based Detection & Segmentation

Dr. Minghan Li

Assistant Professor

Computer Science Department

Colorado School of Mines

Oct/01/2026

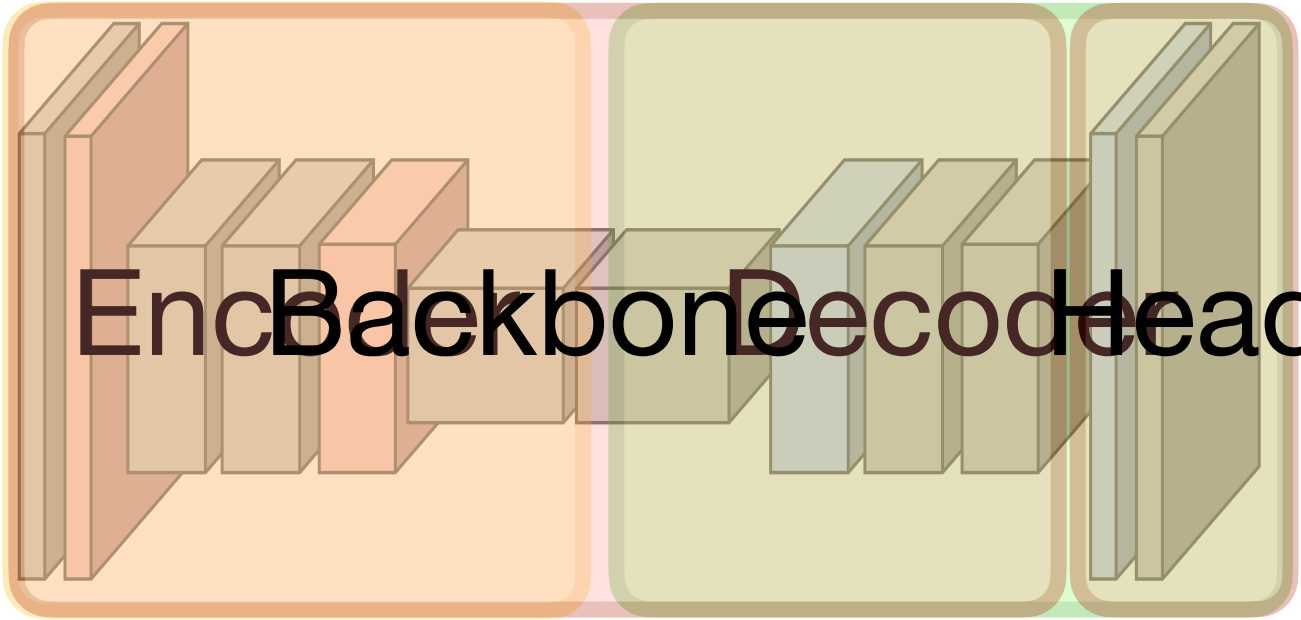


Administration

- N/A

Basic Concepts

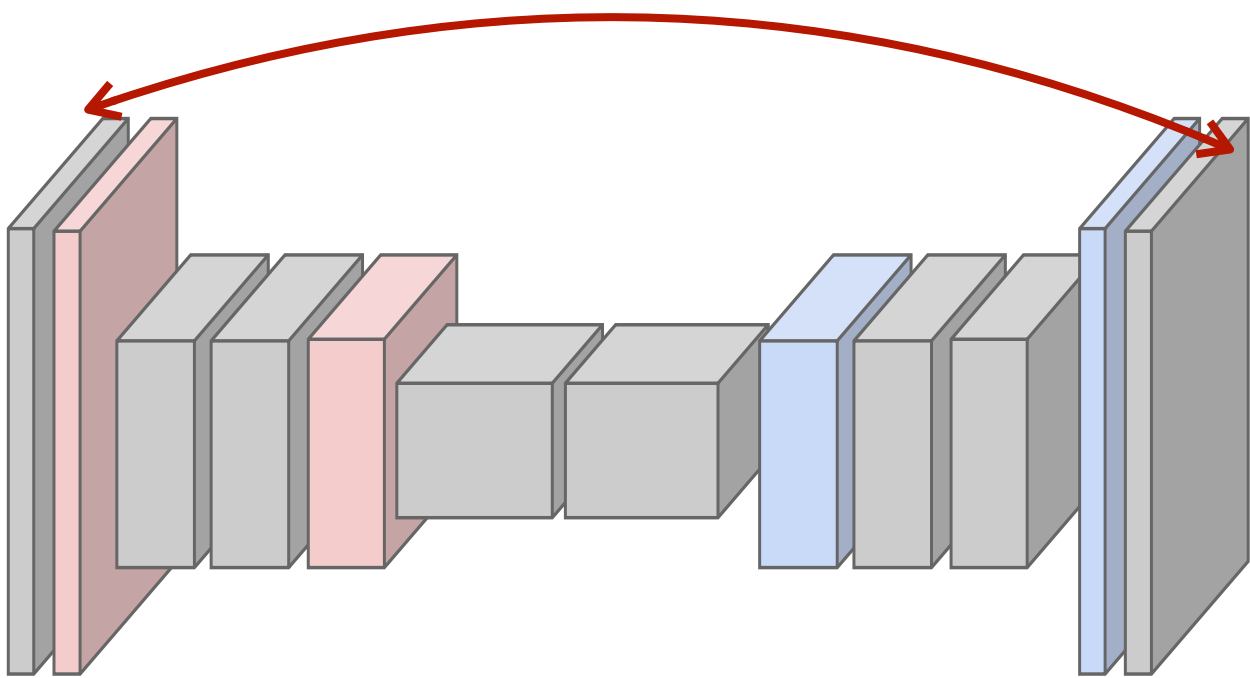
Concept	Simple definition
Encoder	Converts raw input into useful features.
Decoder	Converts features into the desired output.
Backbone	The main network used to extract features.
Head	A task-specific module that produces predictions.
Feature	A learned representation of useful information.
Feature Map	A spatial grid of learned features.
Feature Vector	A vector describing an input or object.
Latent Representation	A compact internal representation of the input.
Patch	A small spatial region of an image.
Patch Embedding	A vector representation of an image patch.
Embedding	A vector representation of information.
Token	A basic unit processed by a Transformer.
Position Embedding	Encodes where a token is located.



Basic Concepts

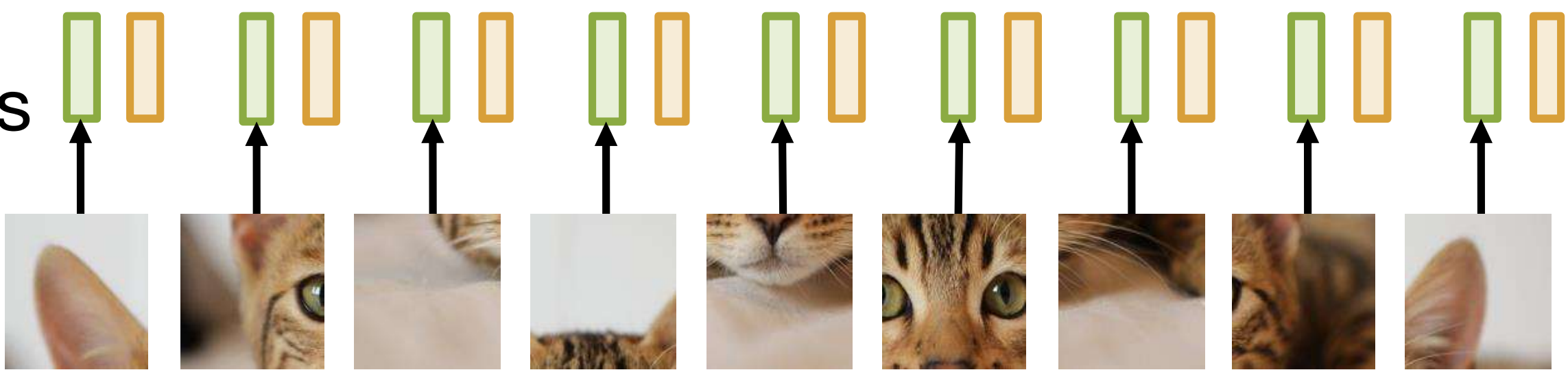
Concept	Simple definition
Encoder	Converts raw input into useful features.
Decoder	Converts features into the desired output.
Backbone	The main network used to extract features.
Head	A task-specific module that produces predictions.
Feature	A learned representation of useful information.
Feature Map	A spatial grid of learned features.
Feature Vector	A vector describing an input or object.
Latent Representation	A compact internal representation of the input.
Patch	A small spatial region of an image.
Patch Embedding	A vector representation of an image patch.
Embedding	A vector representation of information.
Token	A basic unit processed by a Transformer.
Position Embedding	Encodes where a token is located.

Feature / Feature Map
=> Latent Representation



Patch
embeddings

Patches



Word tokens

“I” “love” “this” “class” “!”

Recap

Bounding Box Regression: Multi-Scale Anchors as References

- Anchor central position, width & height:

$$(x_c^a, y_c^a, w^a, h^a)$$

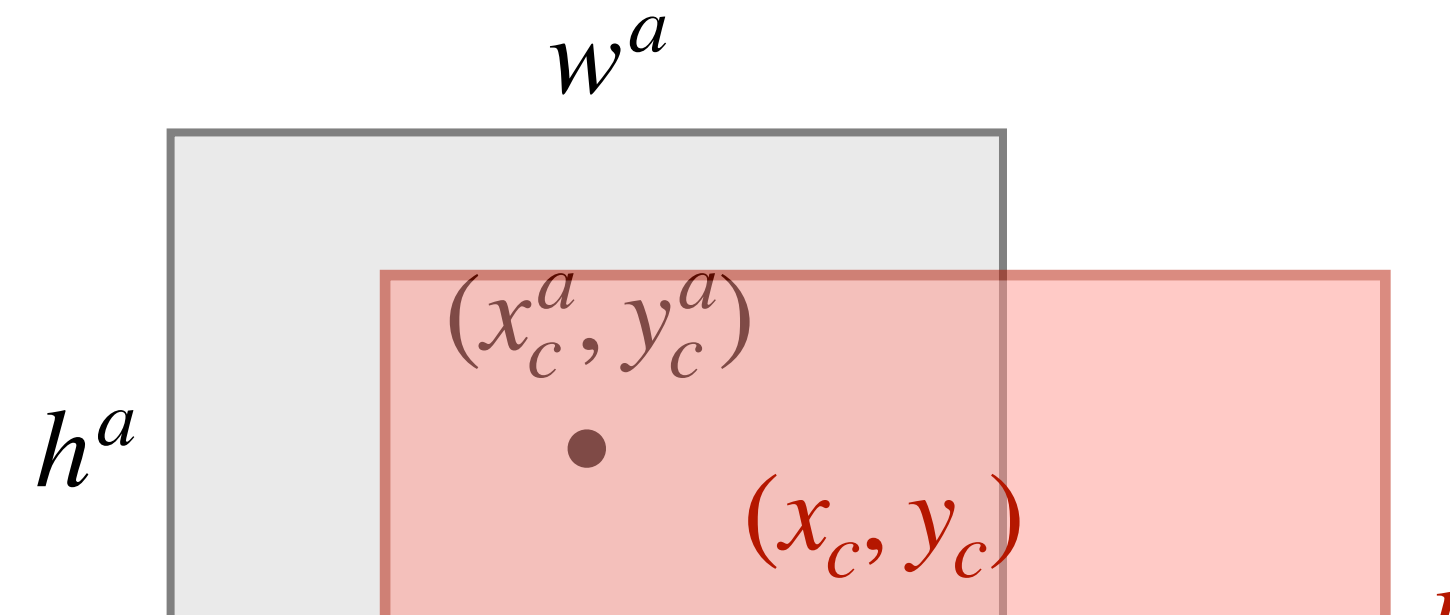
- Ground-truth bounding box:

$$(x_c, y_c, w, h)$$

- Predicted bounding box offsets:

$$(\Delta x, \Delta y, \Delta w, \Delta h)$$

Q: How can we calculate offsets?



Normalizing center shifts by anchor size and predicting log-scale size changes gives similar regression targets for small and large objects, making optimization easier.

$$\Delta w = \log\left(\frac{w}{w^a}\right) \quad \Delta x = \frac{(x_c - x_c^a)}{w^a}$$

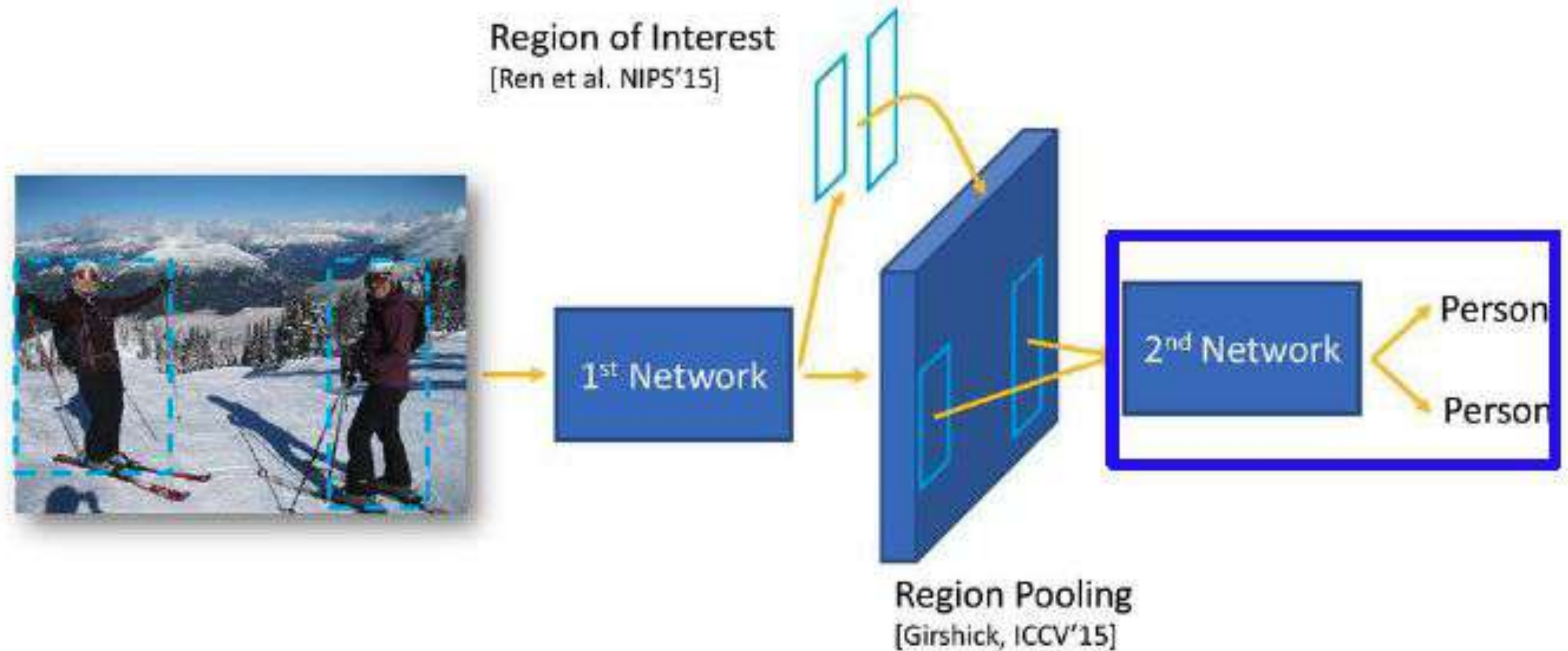
$$\Delta h = \log\left(\frac{h}{h^a}\right) \quad \Delta y = \frac{(y_c - y_c^a)}{h^a}$$

- the model predicts shifts for its center and log-scale changes for its size.

Recap

Details for Two-Stage Object Detectors

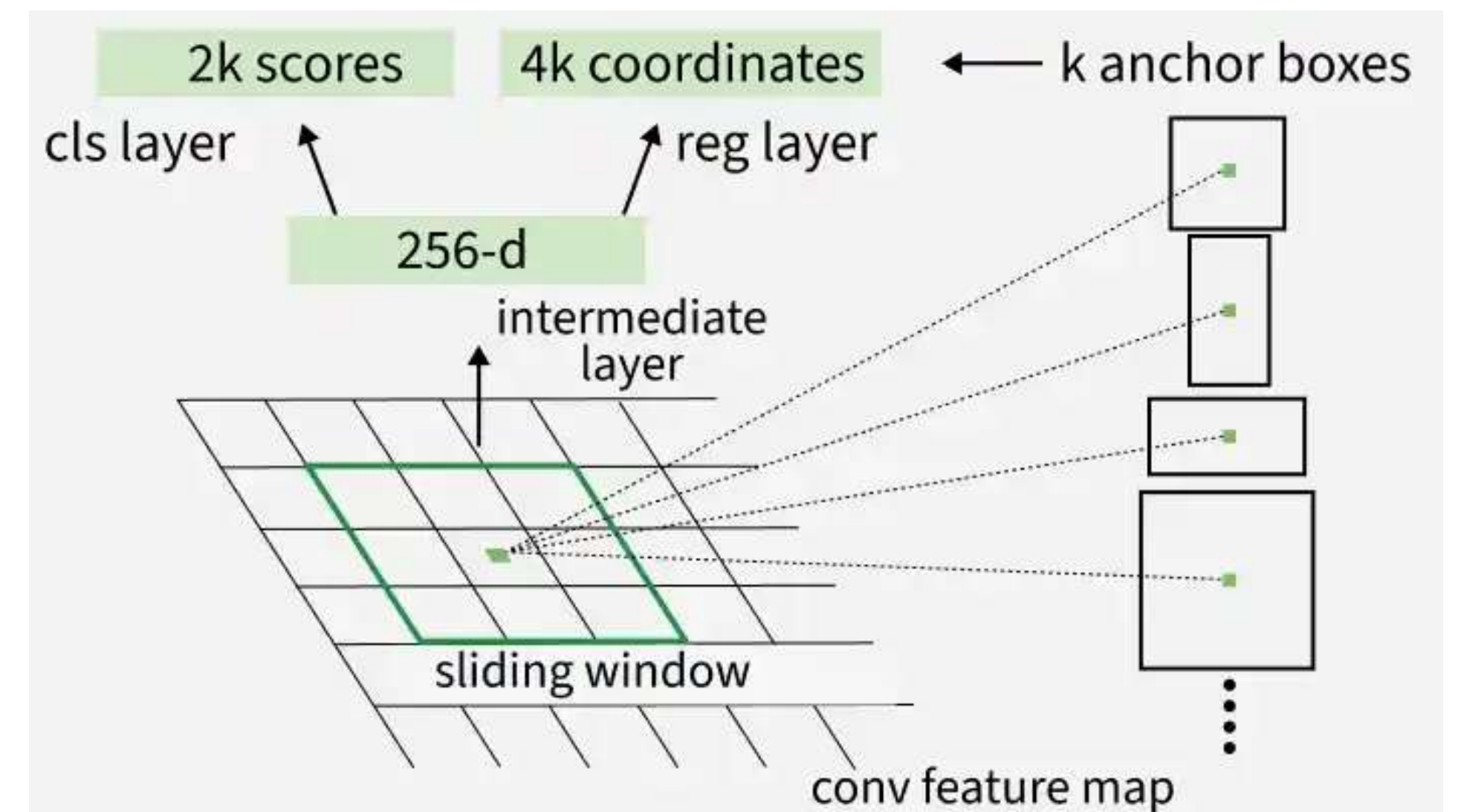
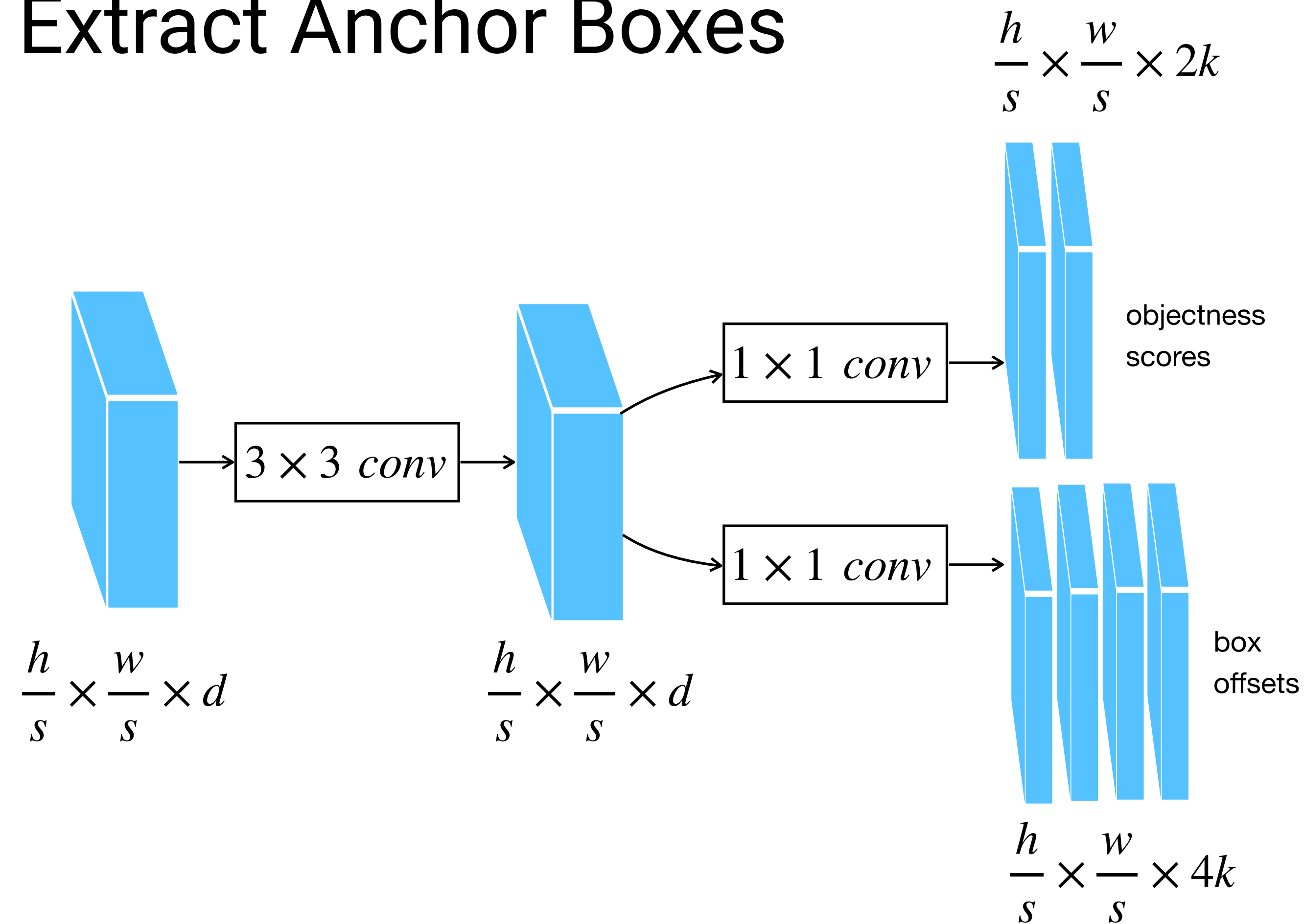
Stage 1: Region Proposal
Network (RPN) (~0.1 sec)



Stage 2: RoI Pooling
(~0.3 sec)

Recap

Extract Anchor Boxes



https://github.com/facebookresearch/Detectron/blob/main/detectron/modeling/generate_anchors.py

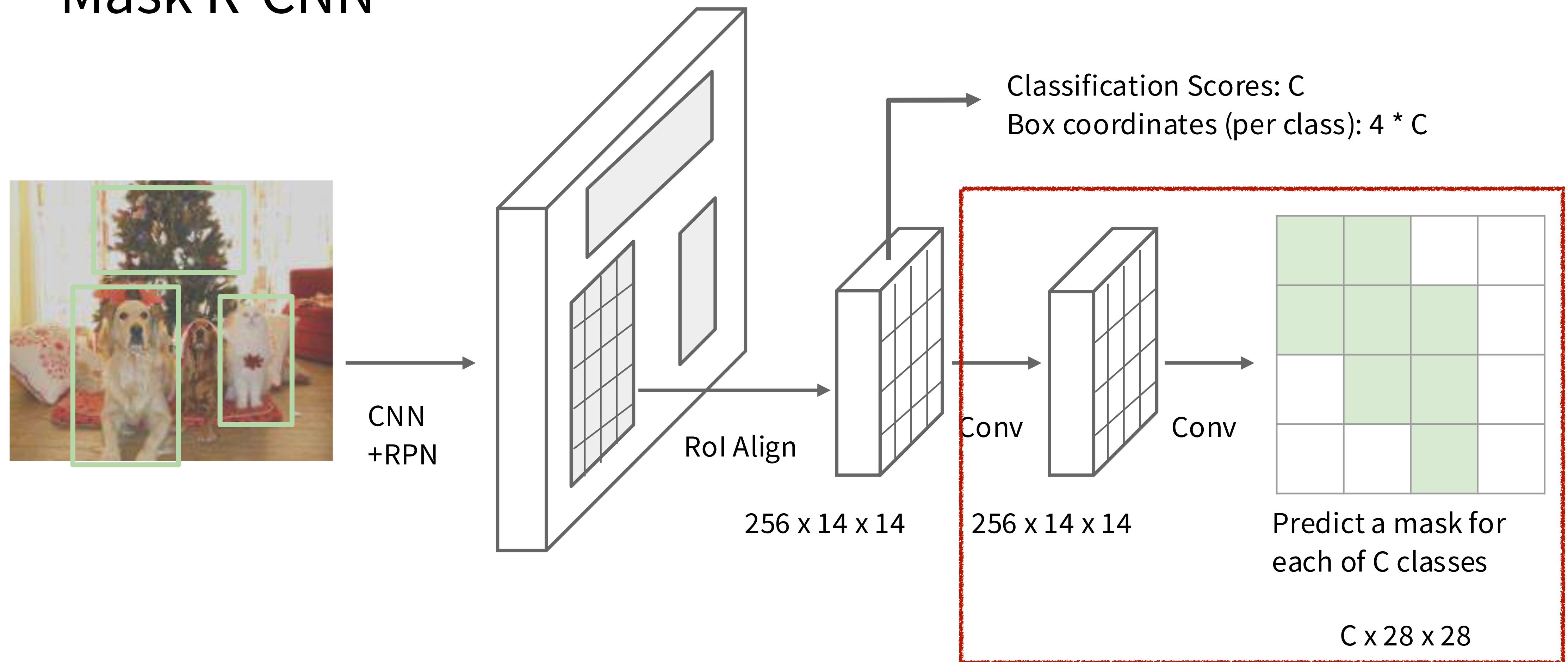
https://github.com/facebookresearch/Detectron/blob/main/detectron/modeling/rpn_heads.py

Recap RoI Align



Recap

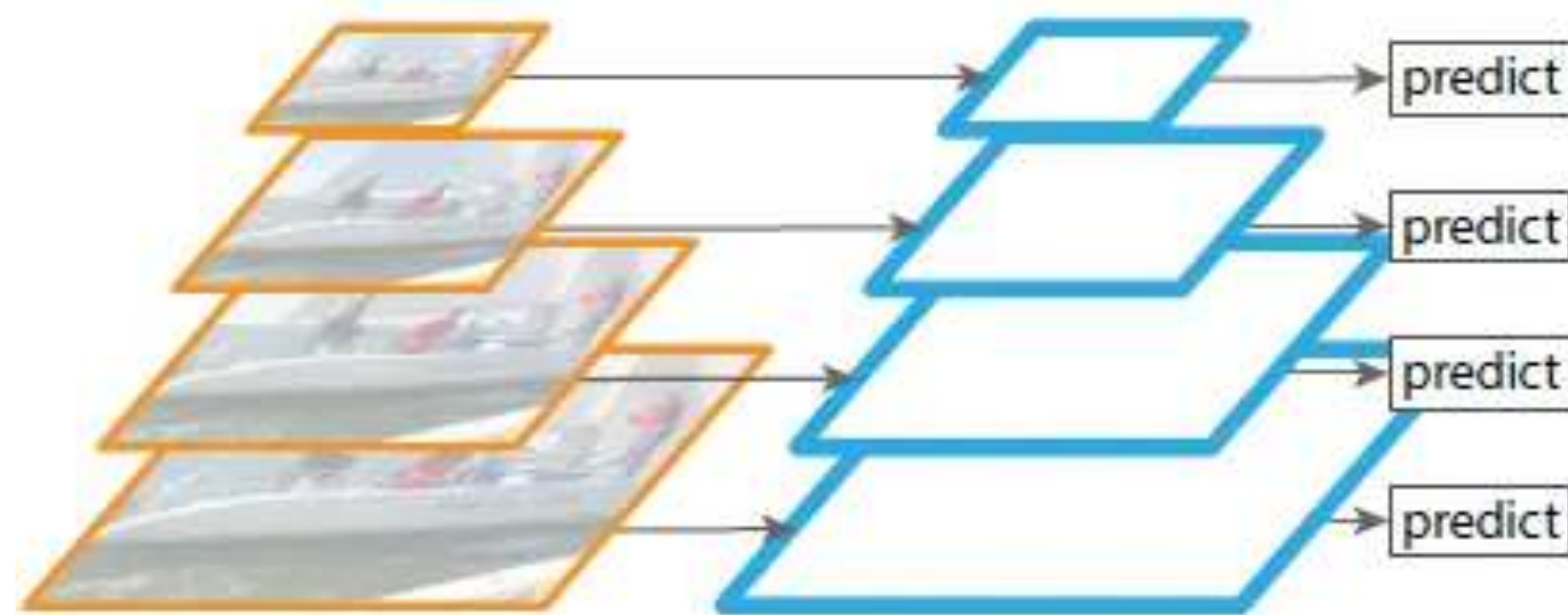
Mask R-CNN



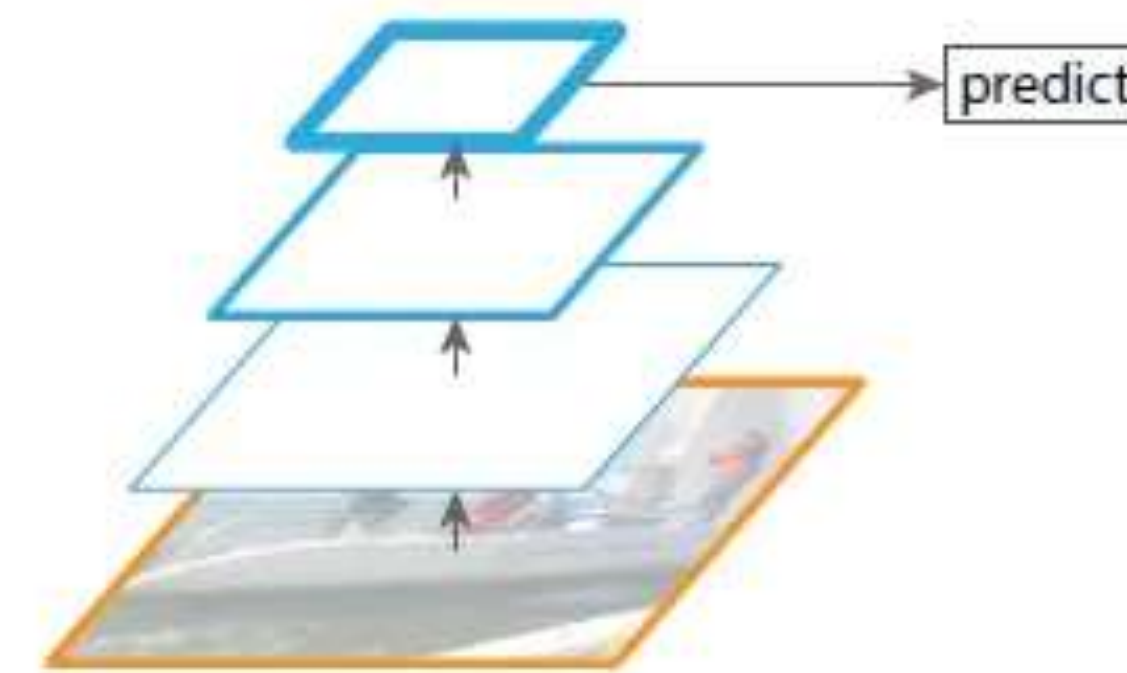
He et al, "Mask R-CNN", arXiv 2017

https://github.com/matterport/mask_RCNN

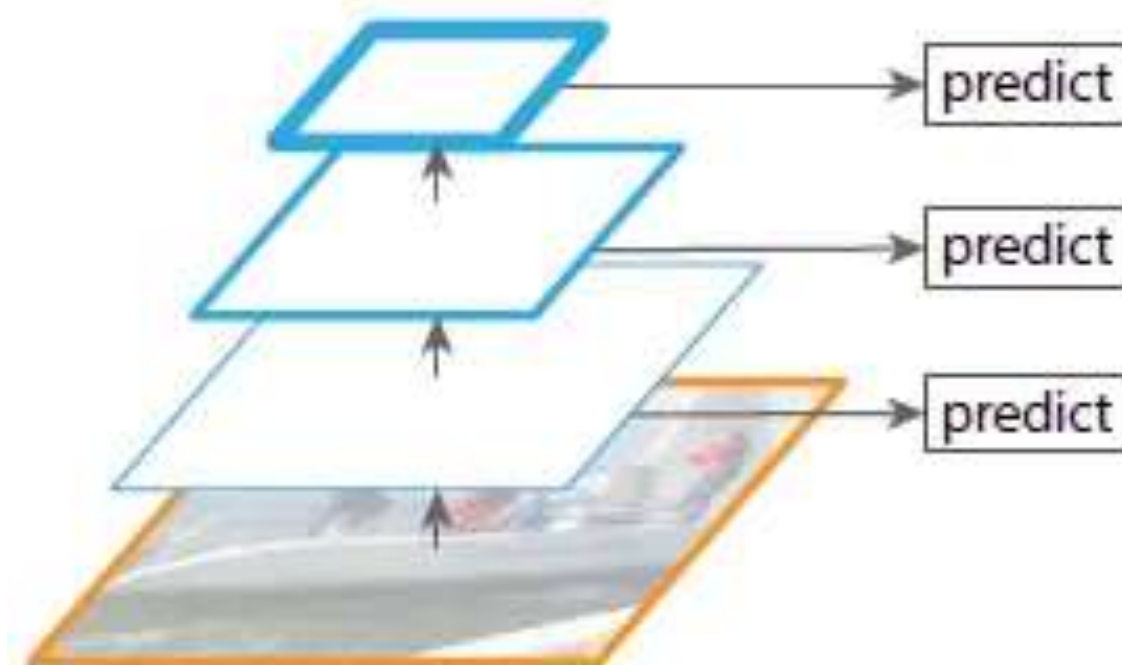
One-stage Object Detector: **YOLO** v.s. **FPN**



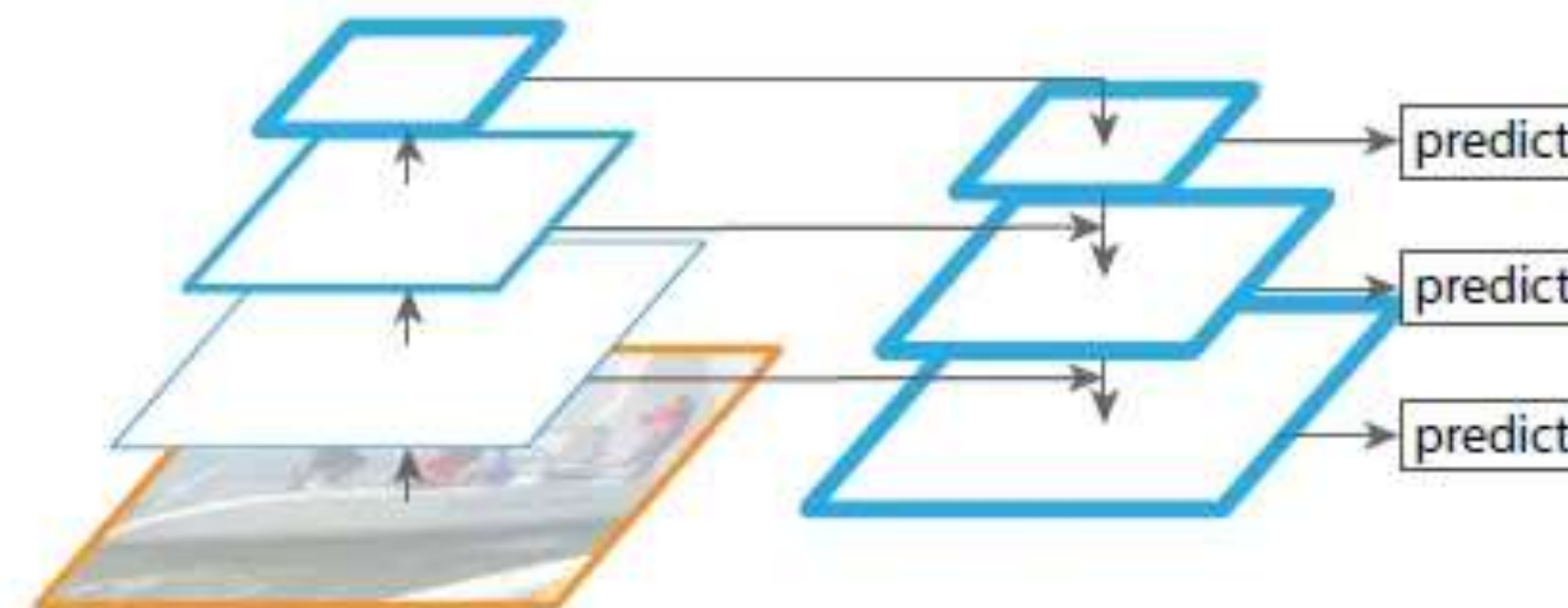
(a) Featurized image pyramid



(b) Single feature map



(c) Pyramidal feature hierarchy



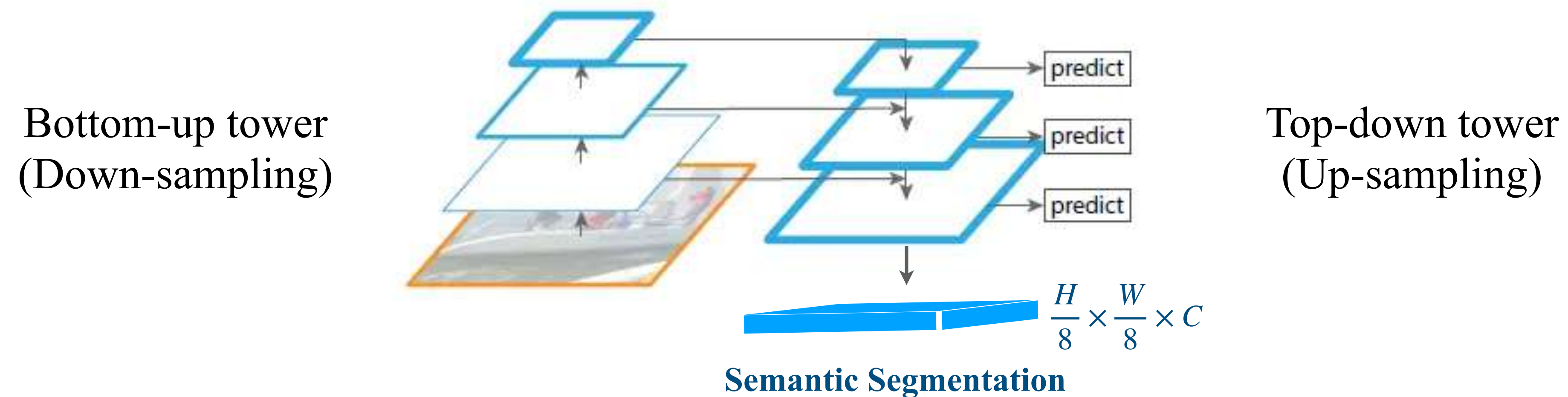
(d) Feature Pyramid Network

Redmon, Joseph, et al. "You only look once: Unified, real-time object detection." *CVPR* 2016.

<https://github.com/ultralytics/ultralytics>

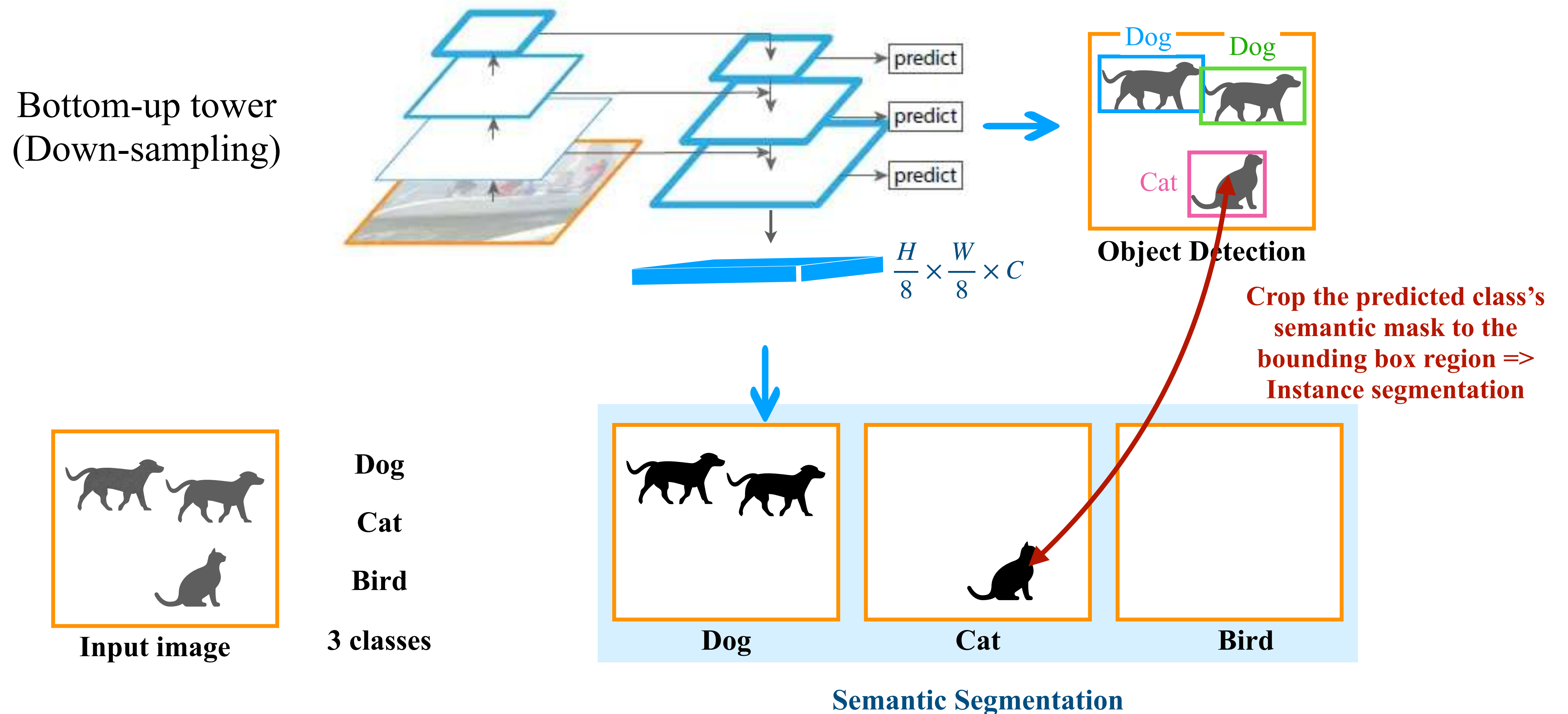
One-stage Instance Segmentation

Object Detection + Semantic Segmentation



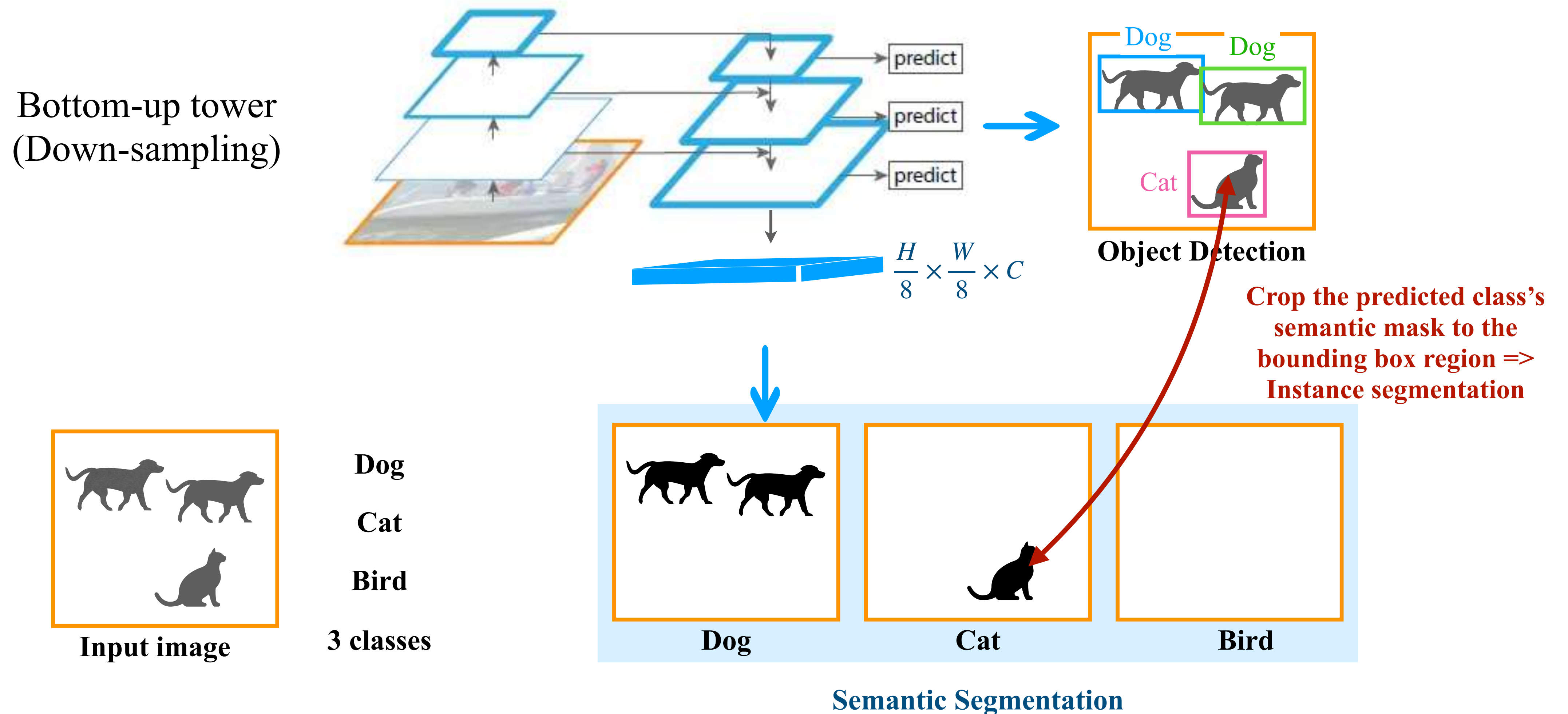
One-stage Instance Segmentation

Object Detection + Semantic Segmentation



One-stage Instance Segmentation

Object Detection + Semantic Segmentation



One-stage Instance Segmentation: YOLACT

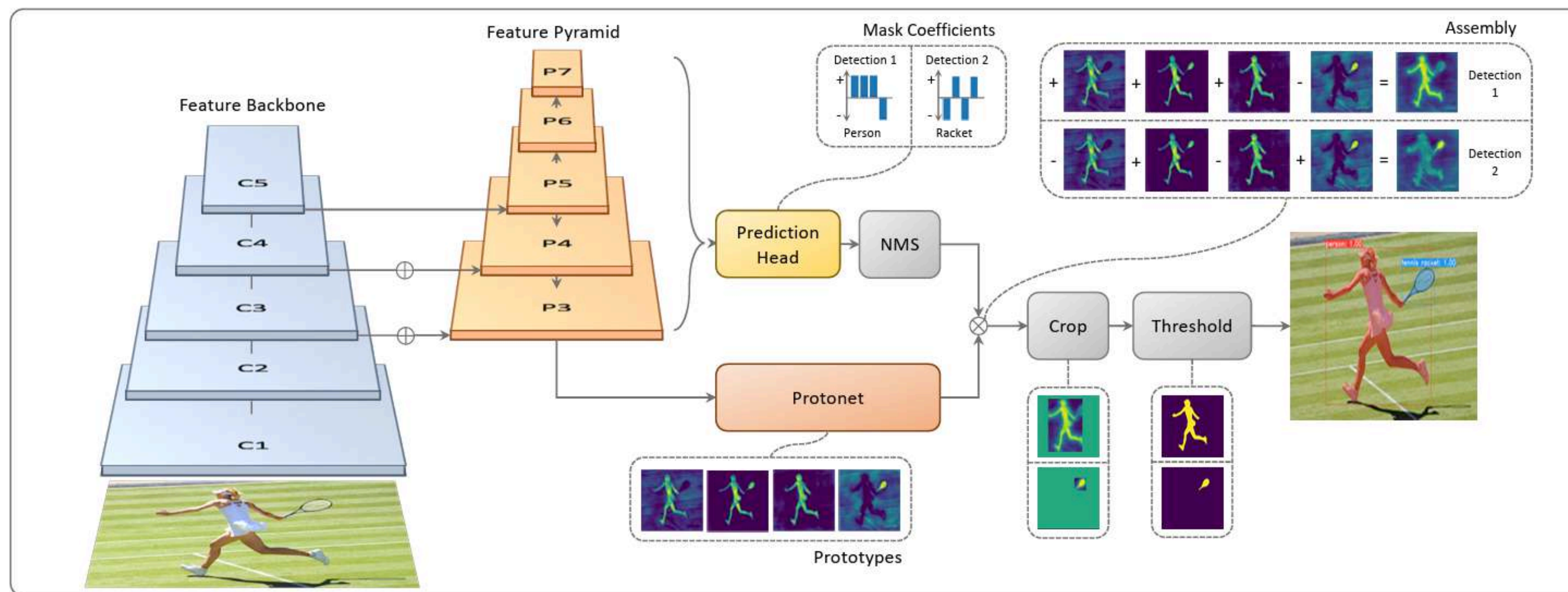
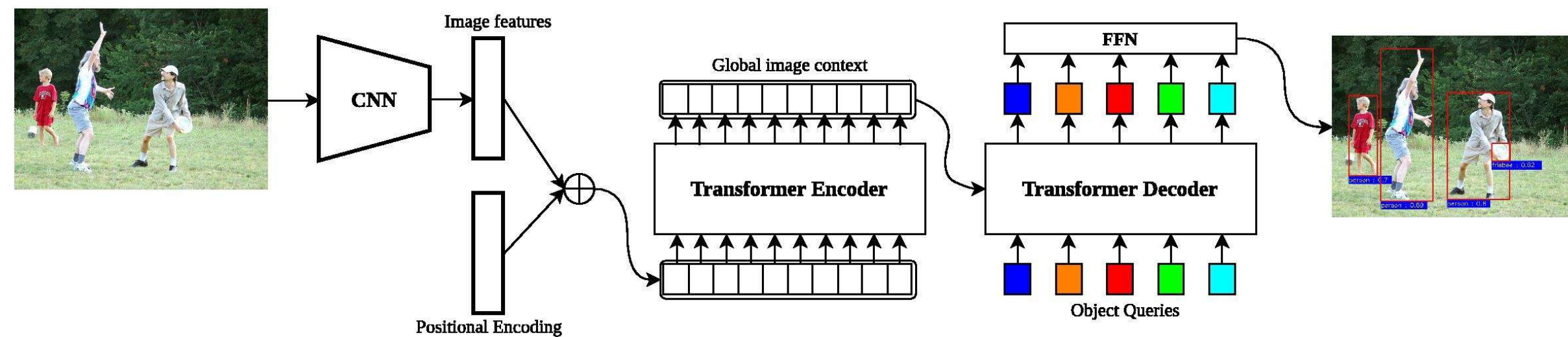


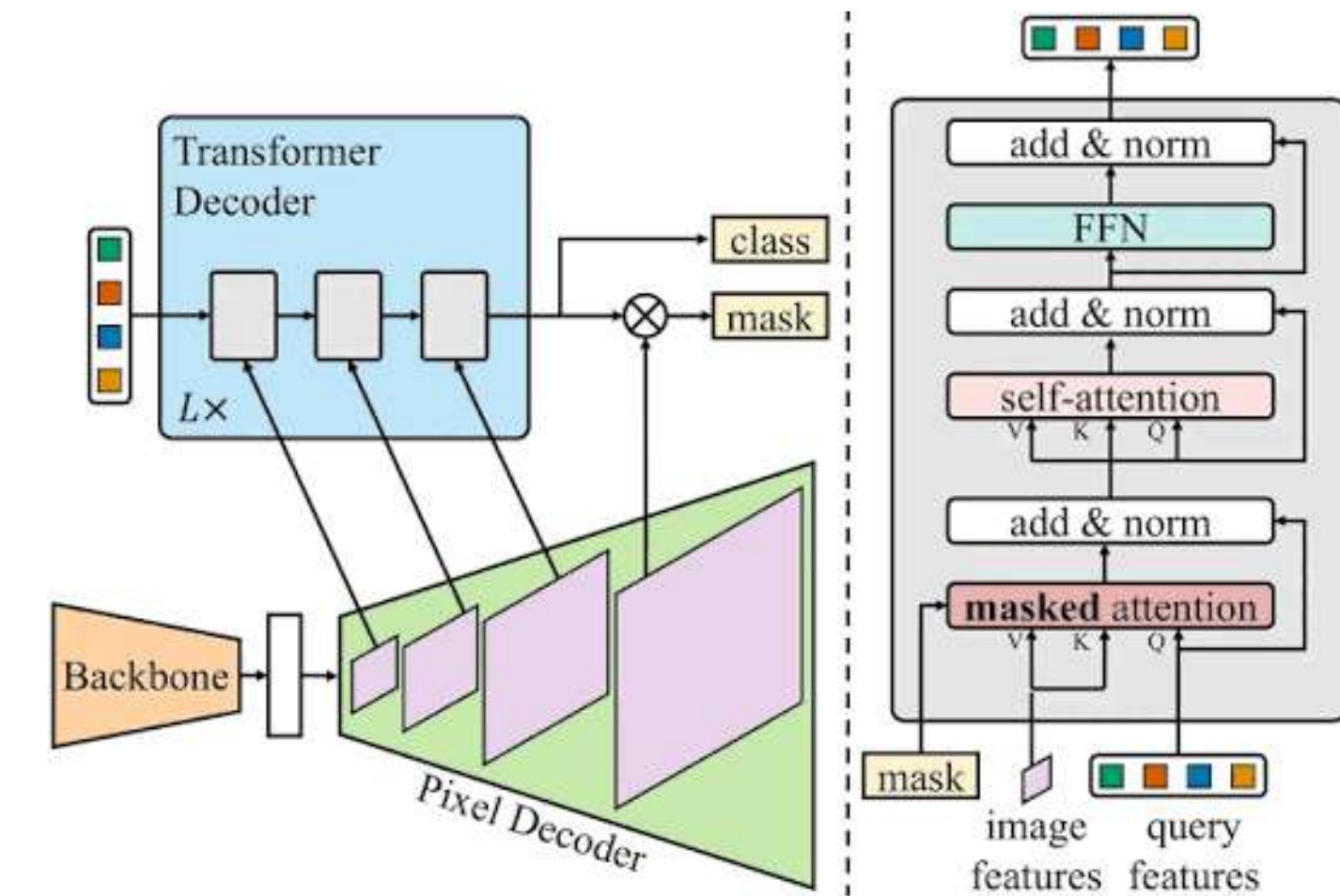
Figure 2: **YOLACT Architecture** Blue/yellow indicates low/high values in the prototypes, gray nodes indicate functions that are not trained, and $k = 4$ in this example. We base this architecture off of RetinaNet [27] using ResNet-101 + FPN.

Today's Agenda

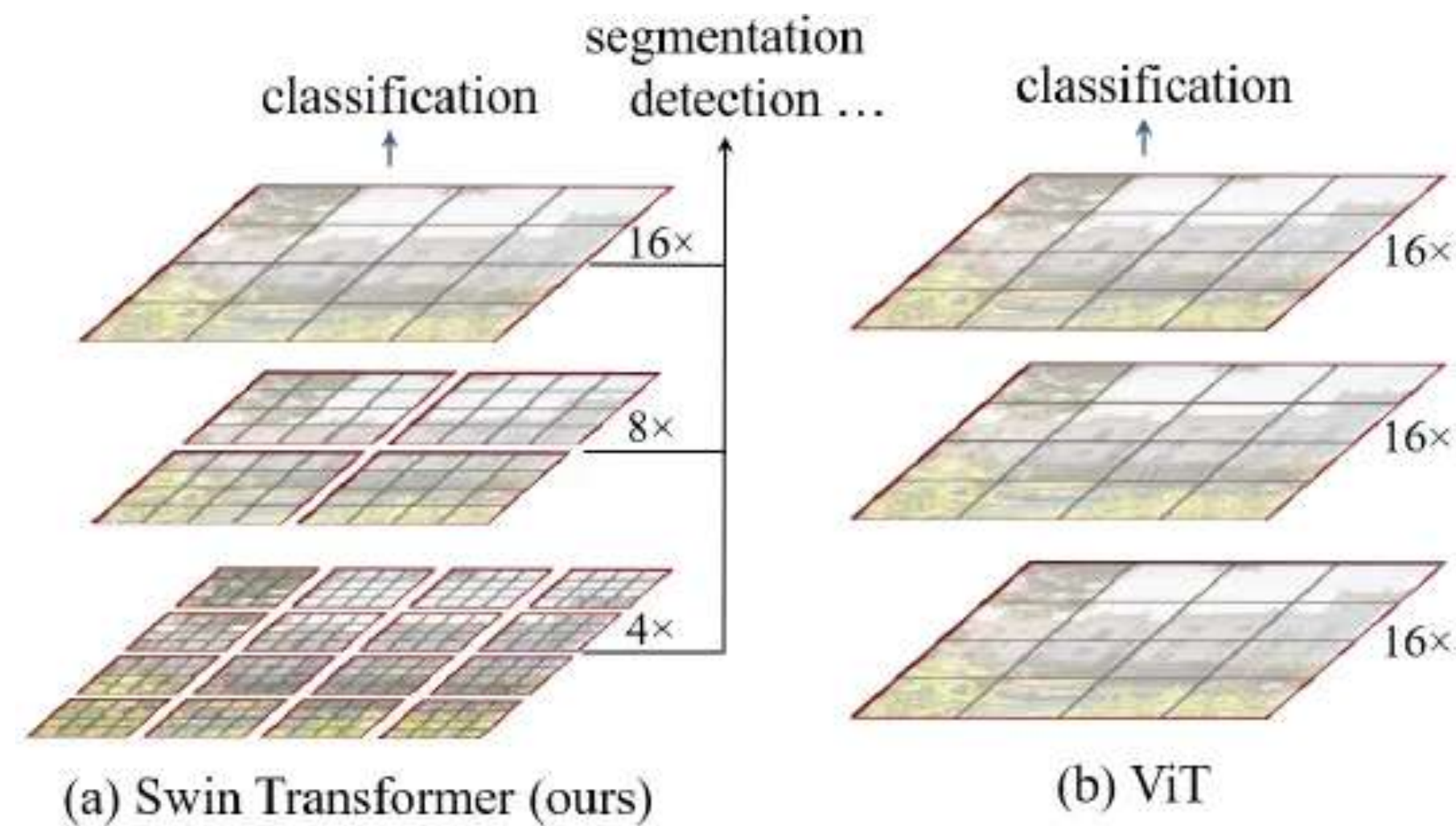
Transformer-based Detection & Segmentation



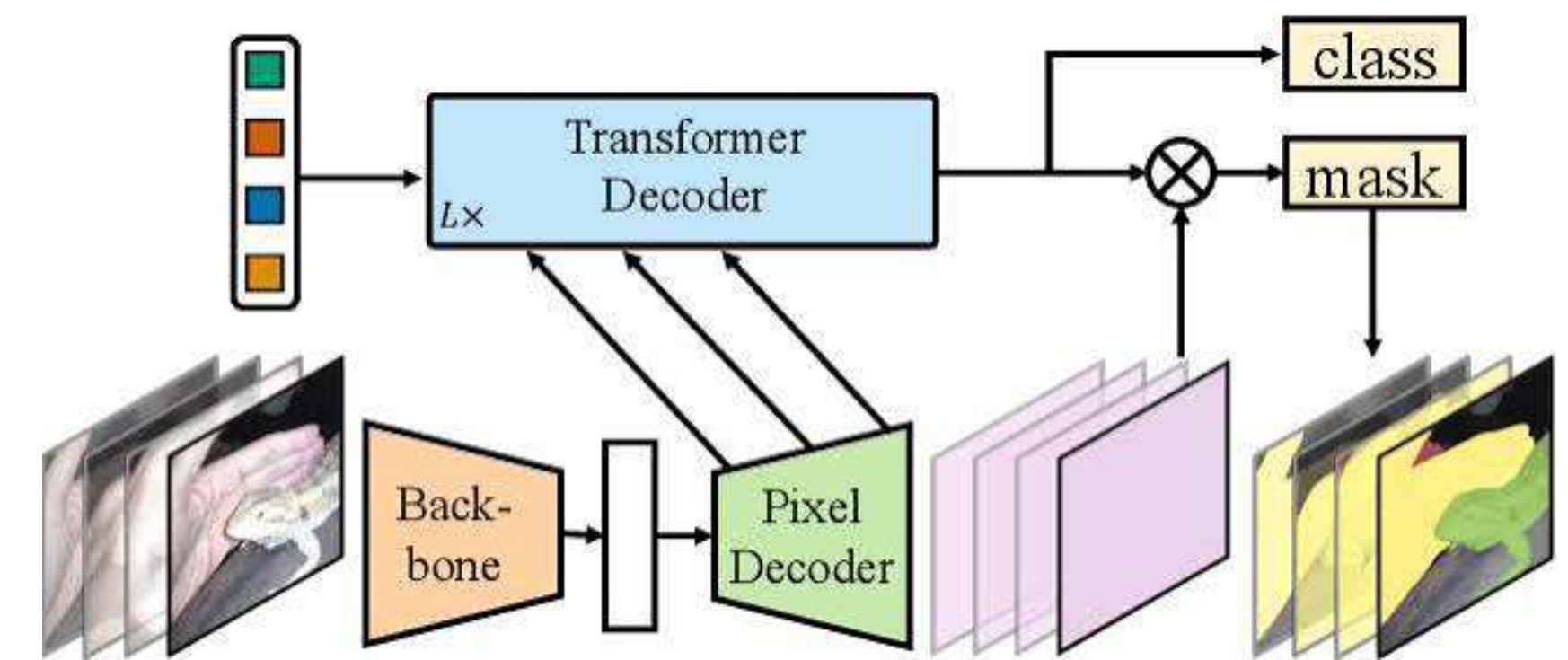
DETR



Mask2Former



Swin Transformer



Mask2Former-Video

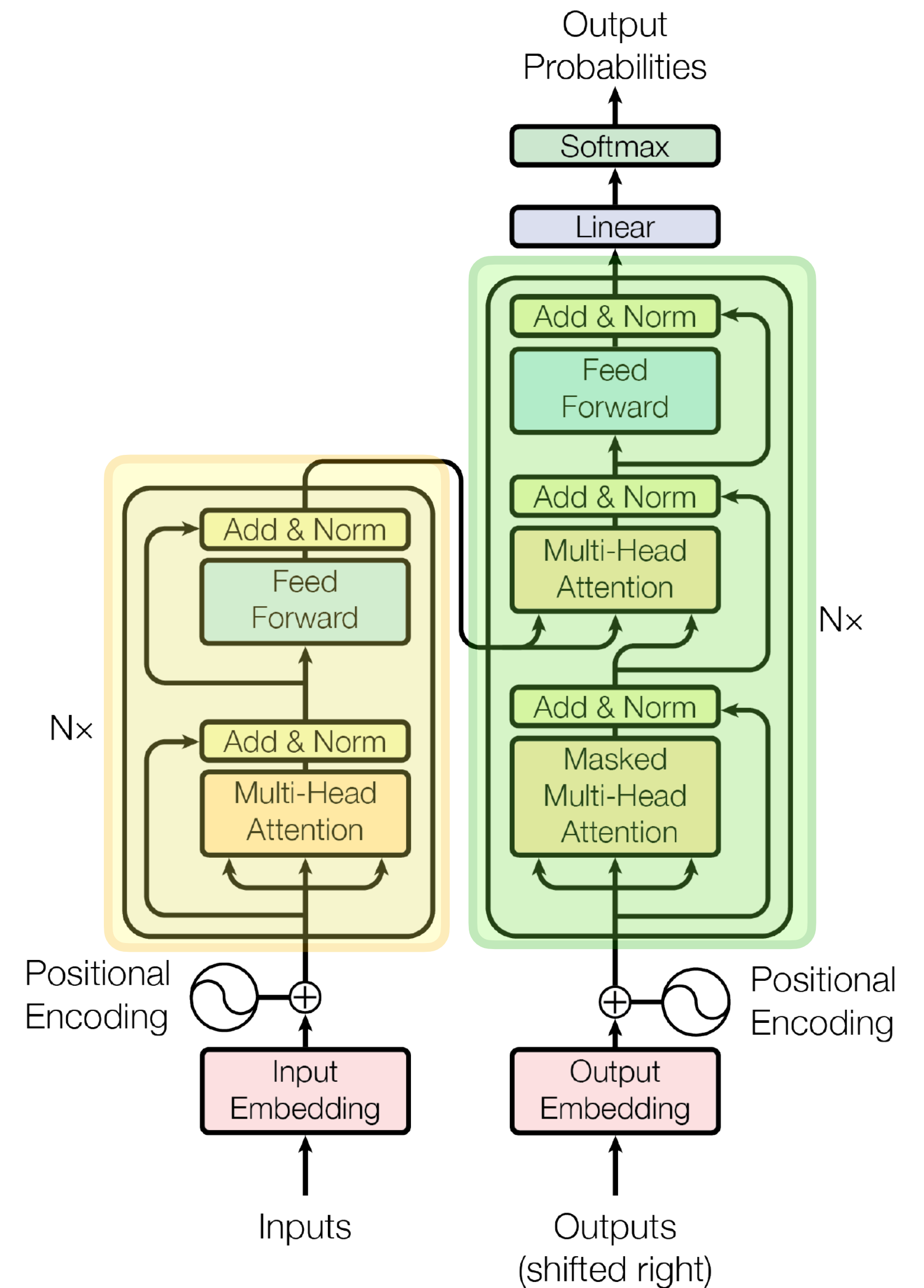
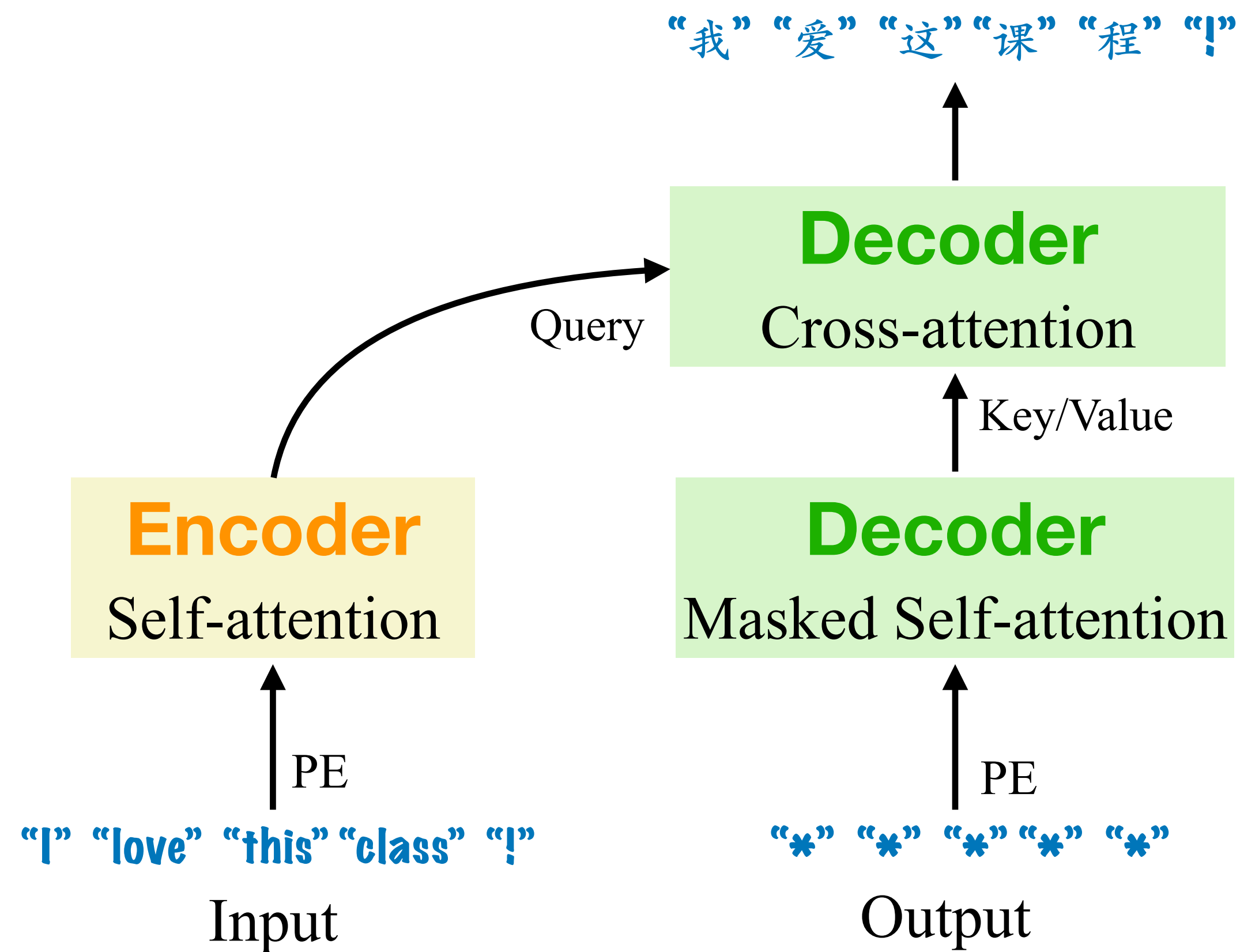
Transformer-based Detection & Segmentation

DETR v.s. Mask2Former

Recap

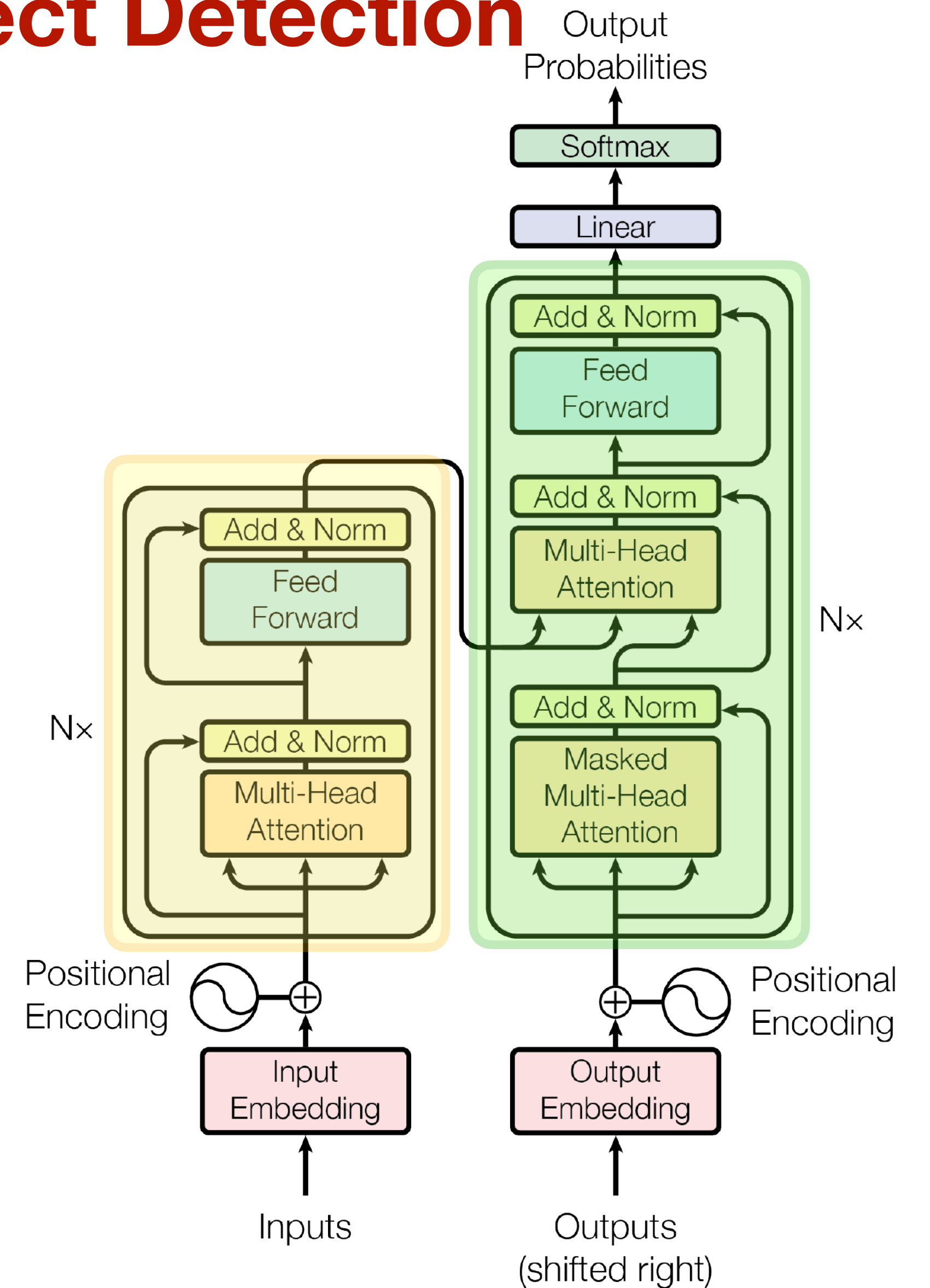
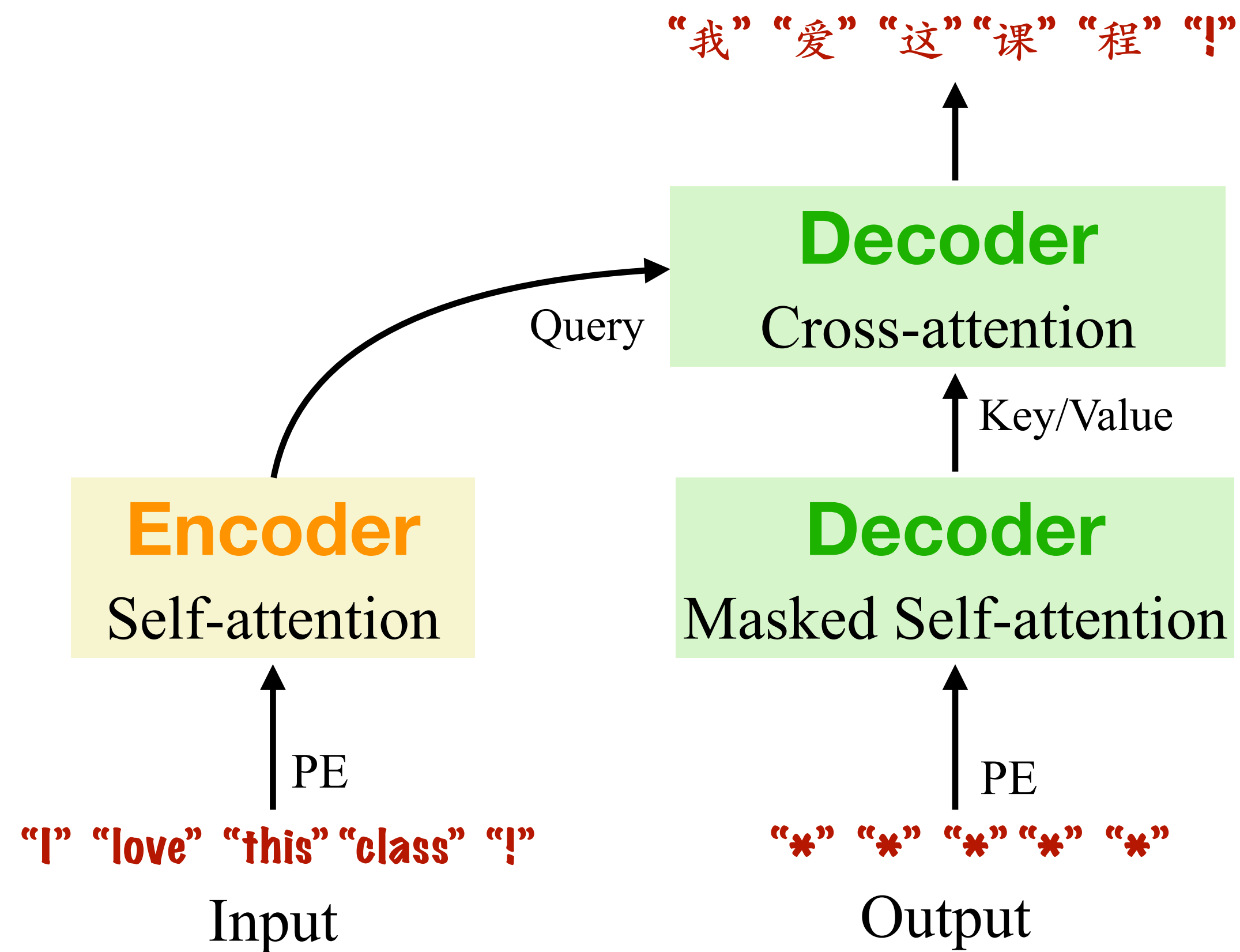
Transformer Model Architecture

Translation task: English $\rightarrow$ Chinese



Transformer Model Architecture for **Object Detection**

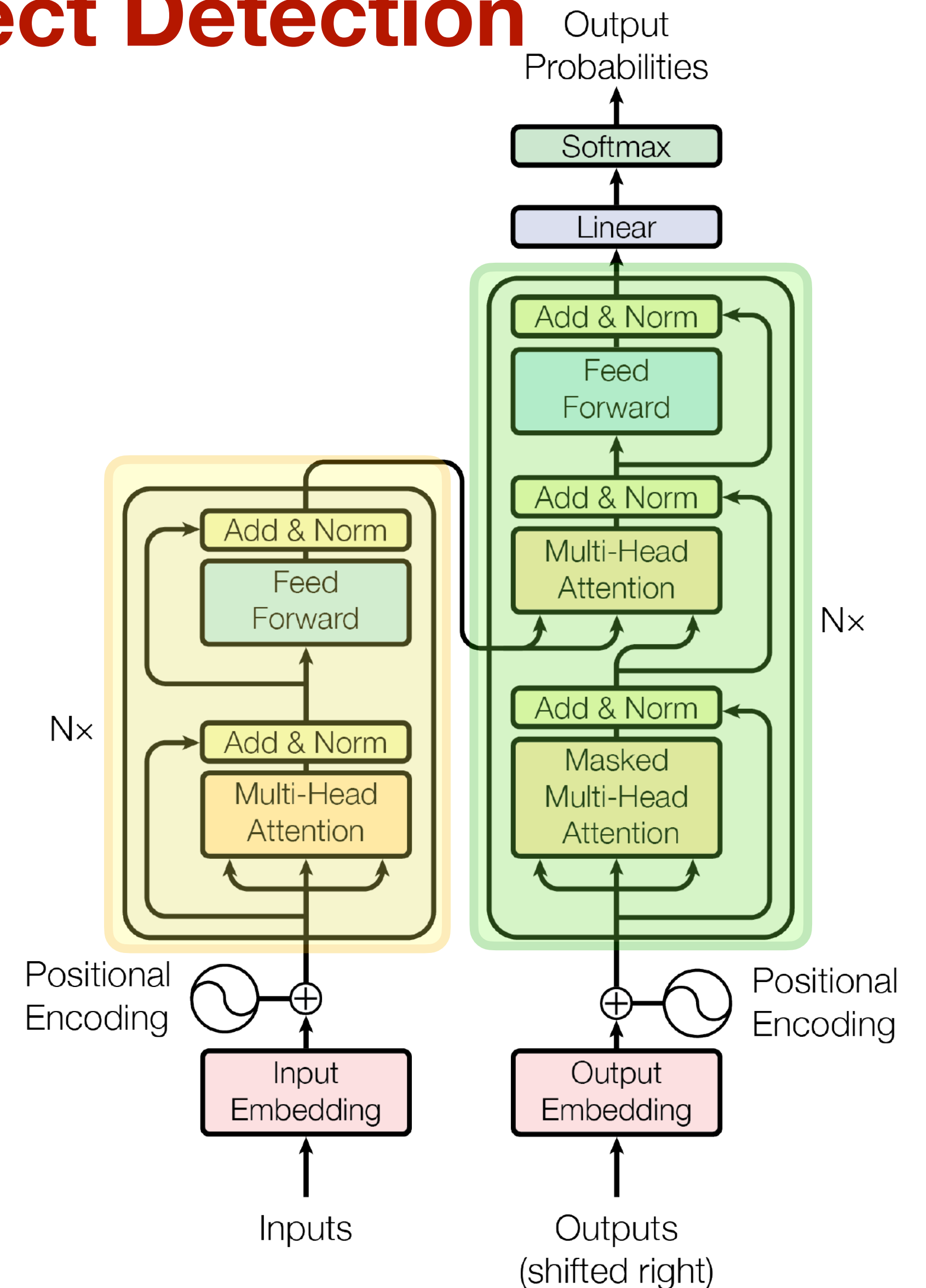
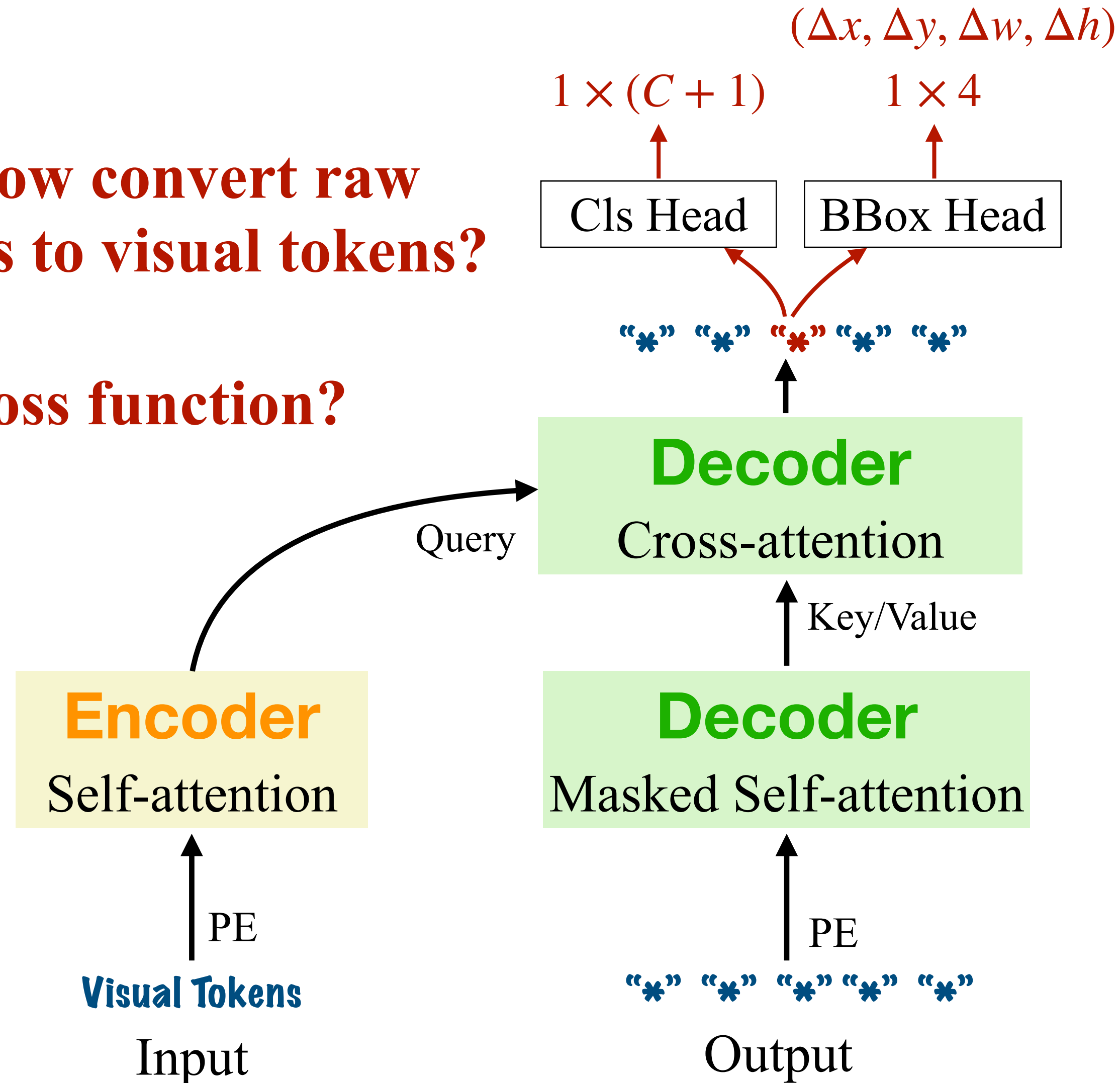
What is the input and output ???



Transformer Model Architecture for **Object Detection**

Q1: How convert raw images to visual tokens?

Q2: Loss function?

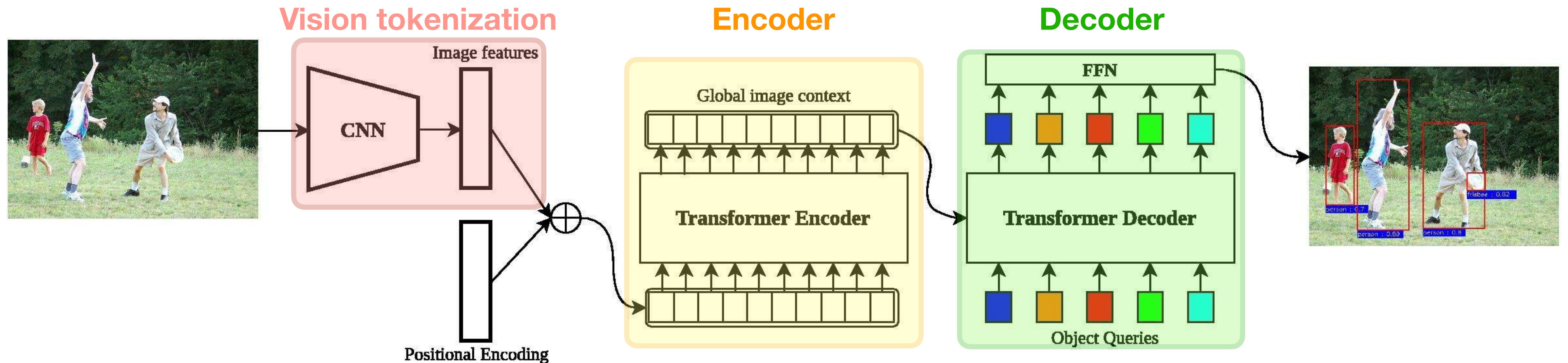


Object **D**etection with **T**ransformers: DETR

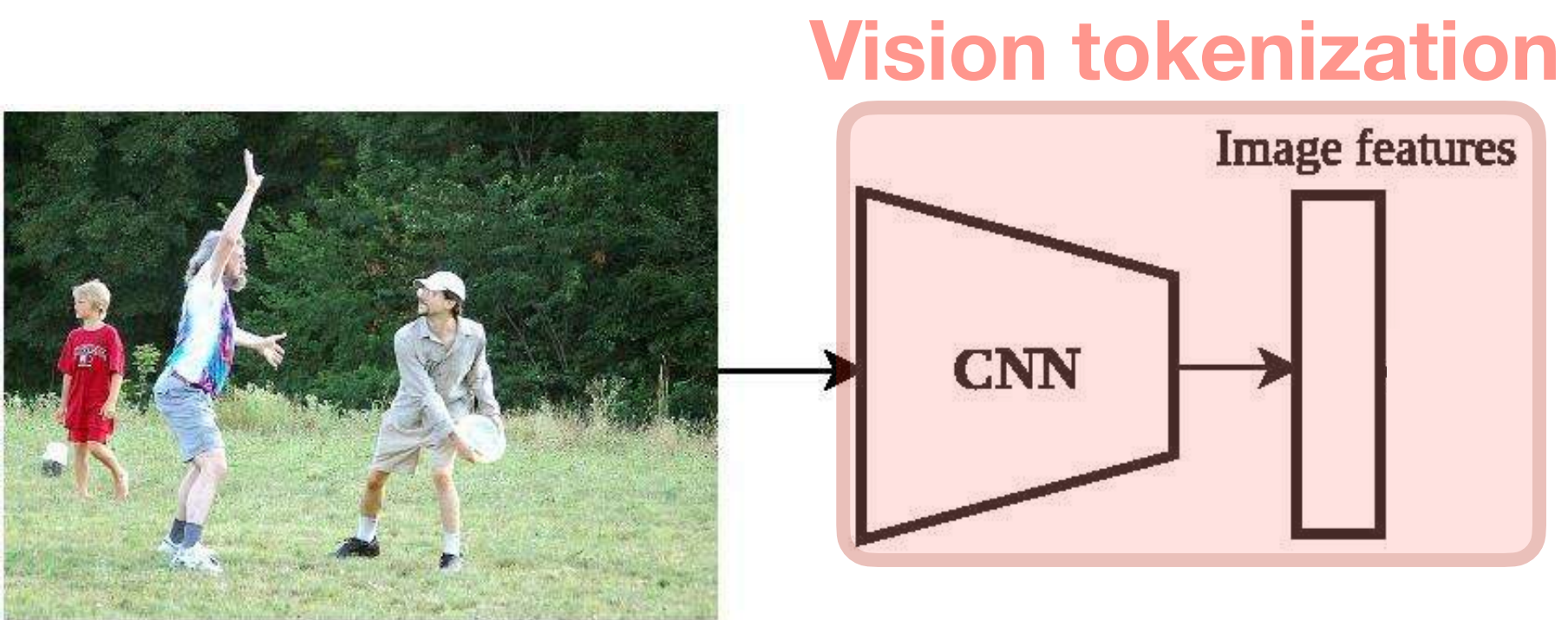
Simple object detection pipeline: directly output a set of boxes from a Transformer

No anchors, no regression of box transforms

Match predicted boxes to GT boxes with bipartite matching; train to regress box coordinates



Object **D**etection with **T**ransformers: DETR

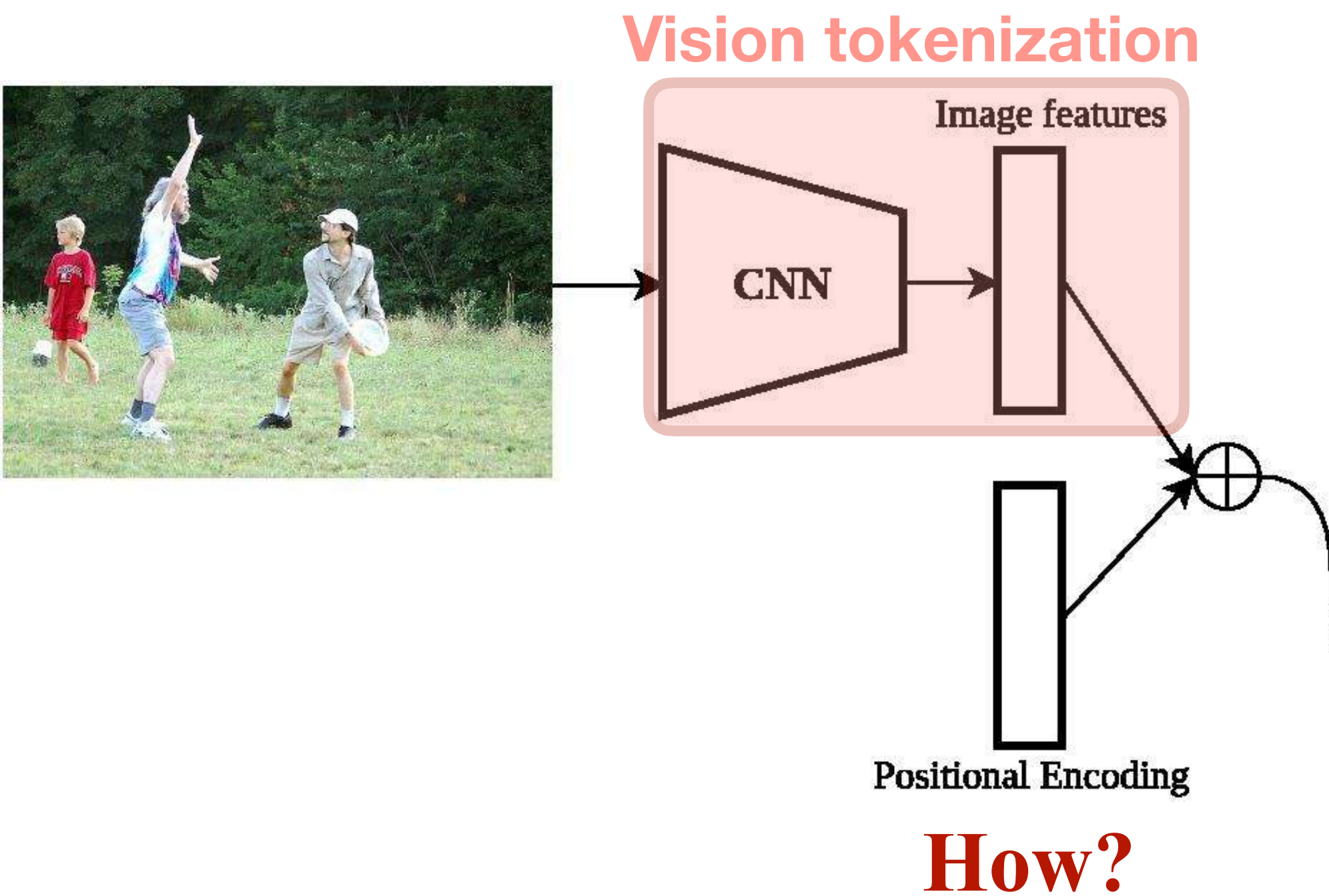


CNN Backbone $\Rightarrow$ Visual Tokens

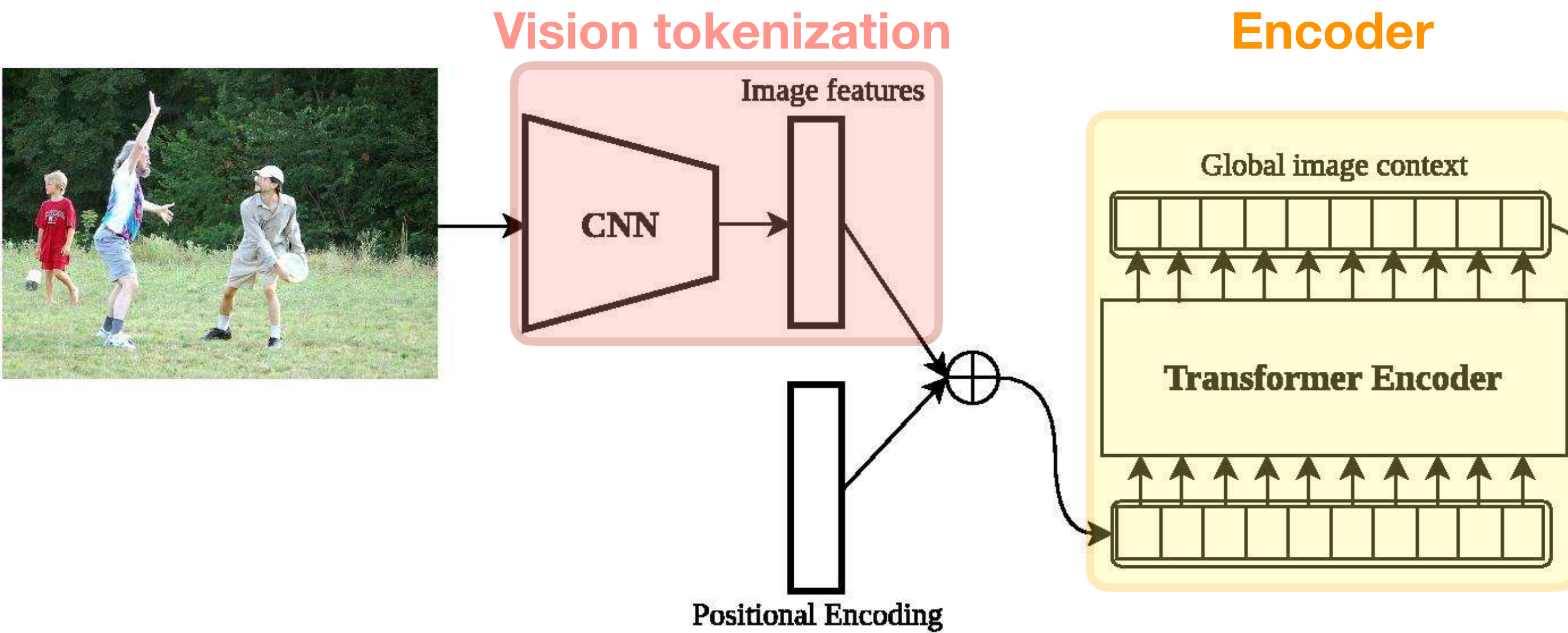
$\Rightarrow$ Stage 0: Feature Extractor

**Q1: How convert raw
images to visual tokens?**

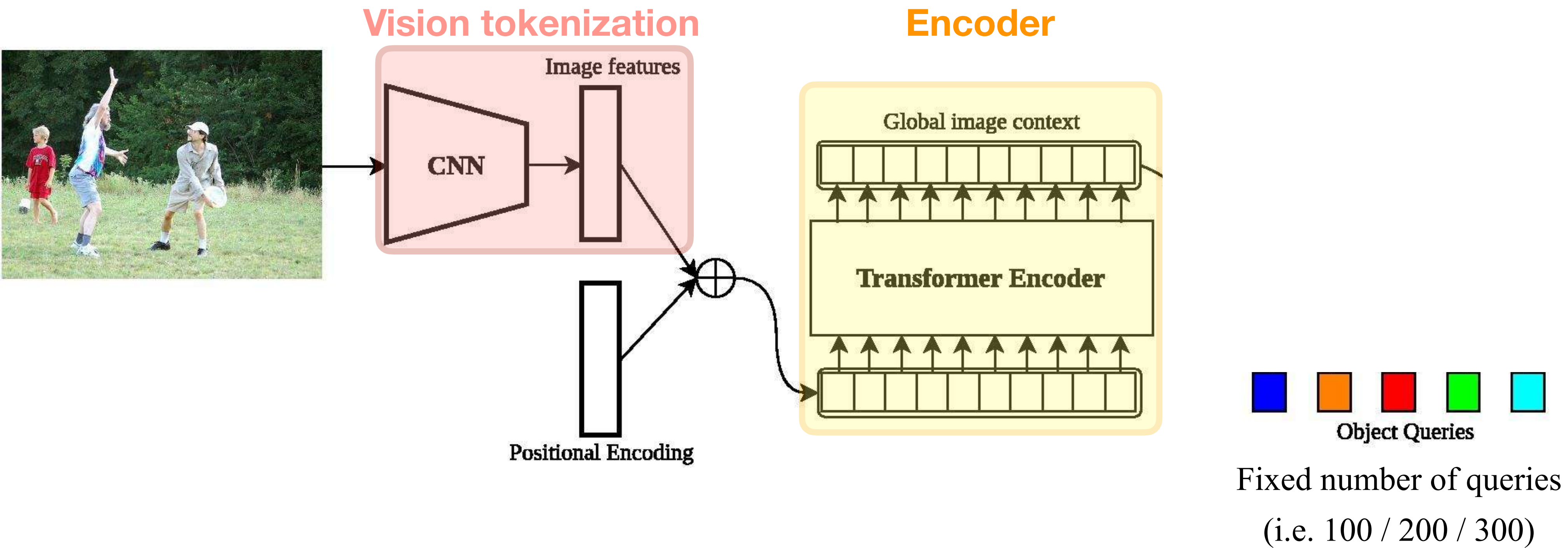
Object **D**etection with **T**ransformers: DETR



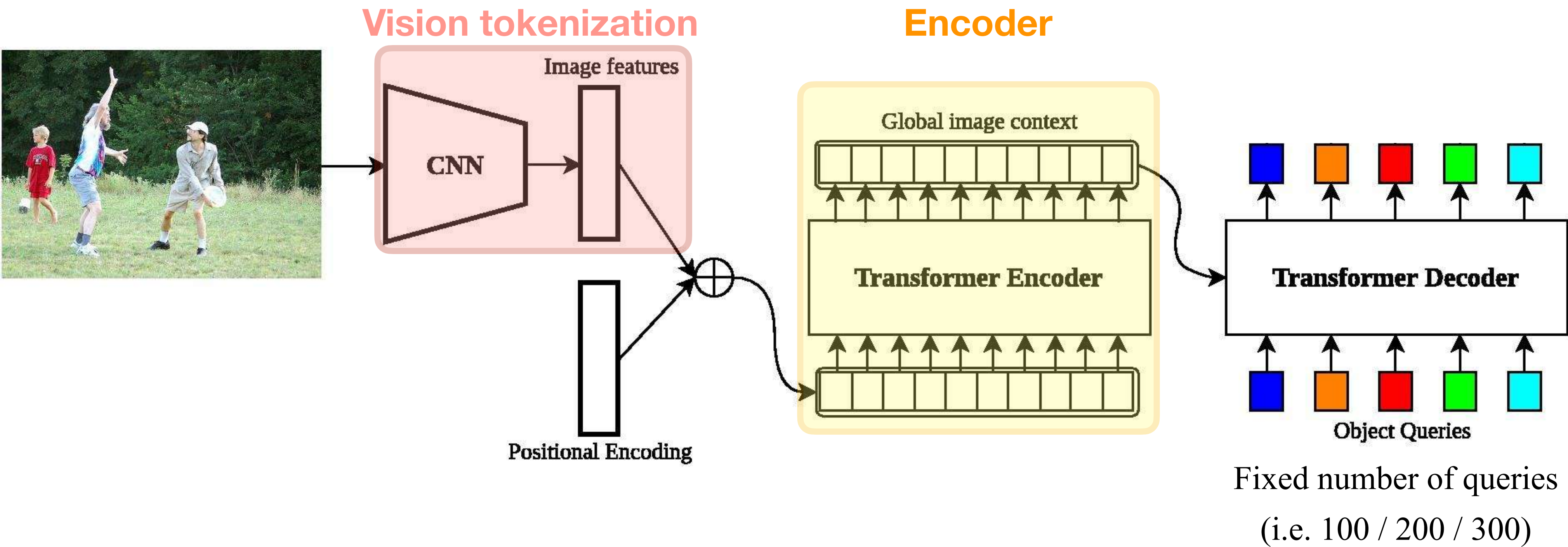
Object **D**etection with **T**ransformers: DETR



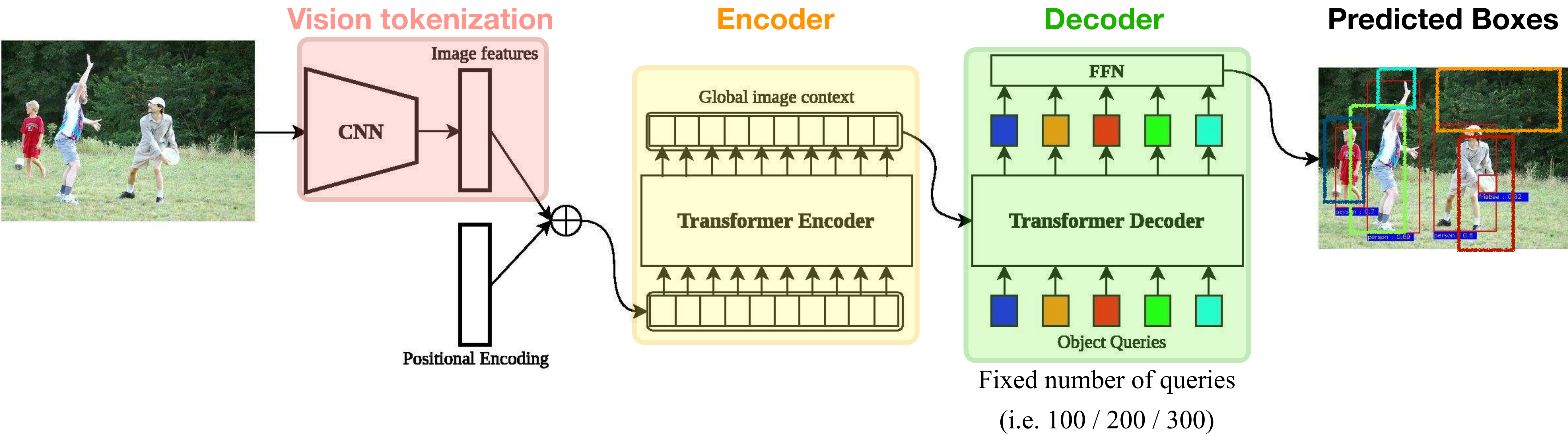
Object **D**etection with **T**ransformers: DETR



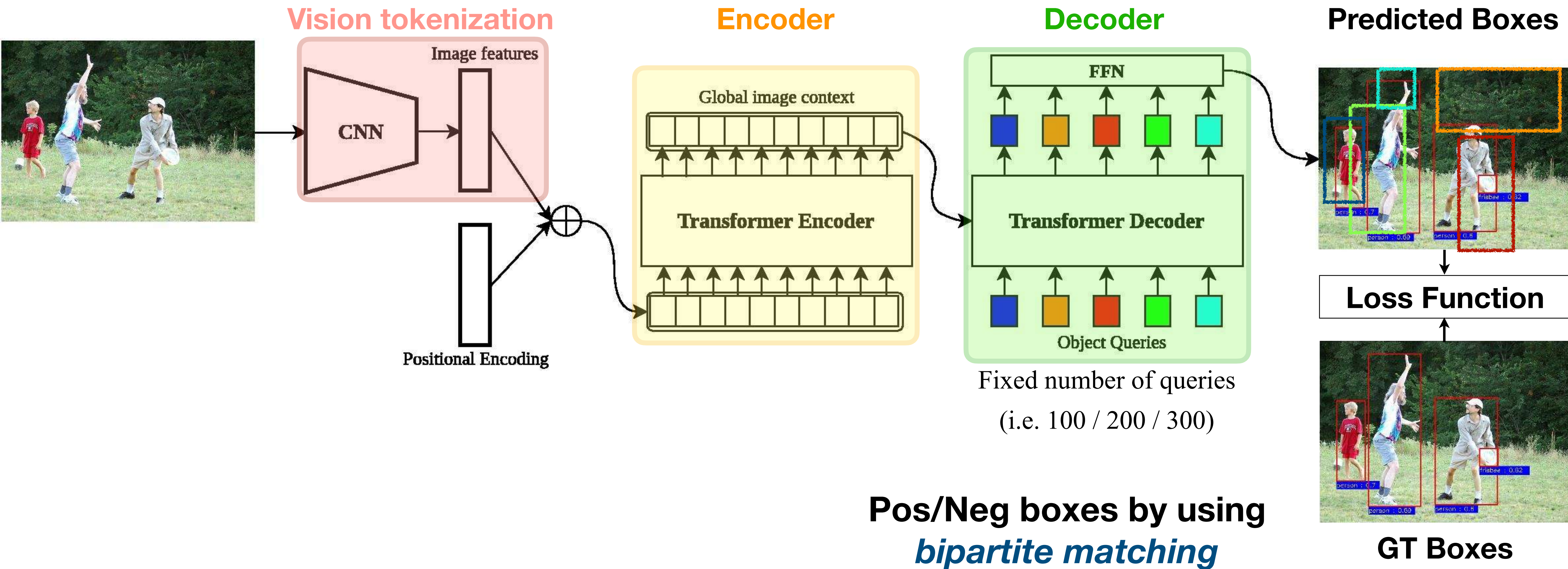
Object **D**etection with **T**ransformers: DETR



Object **D**etection with **T**ransformers: DETR



Object **D**etection with **T**ransformers: DETR



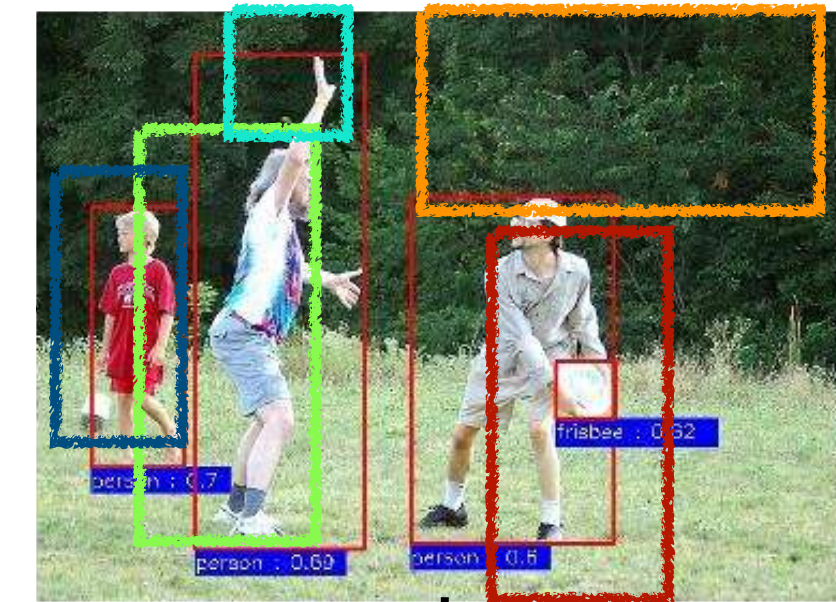
Object **D**etection with **T**ransformers: DETR

DETR assigns positive and negative predictions through

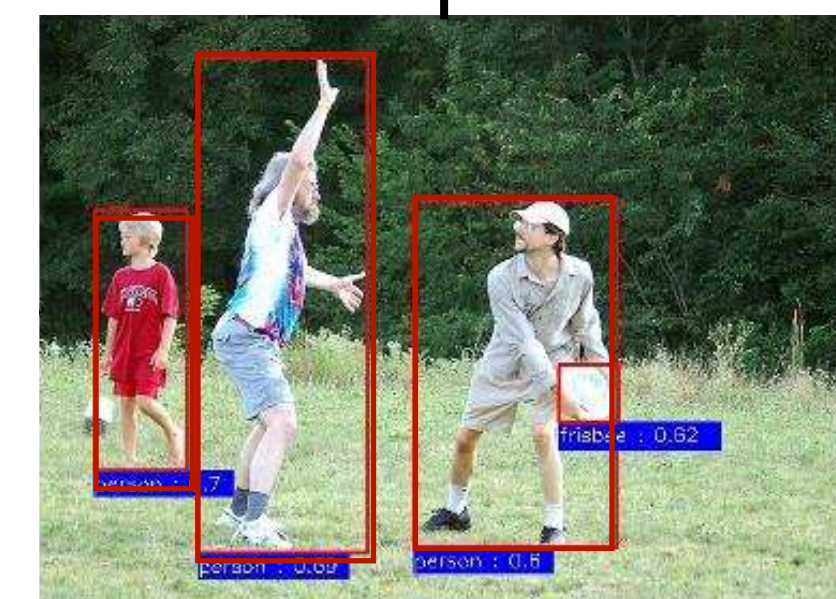
- one-to-one bipartite matching
- solved using the Hungarian algorithm

1. Compute a matching cost for every prediction–ground-truth pair
 - class probability,
 - L1 box distance,
 - GIoU (Generalized Intersection over Union)
2. Use the Hungarian algorithm to find the one-to-one assignment with the lowest total cost.
3. Treat the matched predictions as positives and the unmatched predictions as negatives, labeled `no-object`

Predicted Boxes

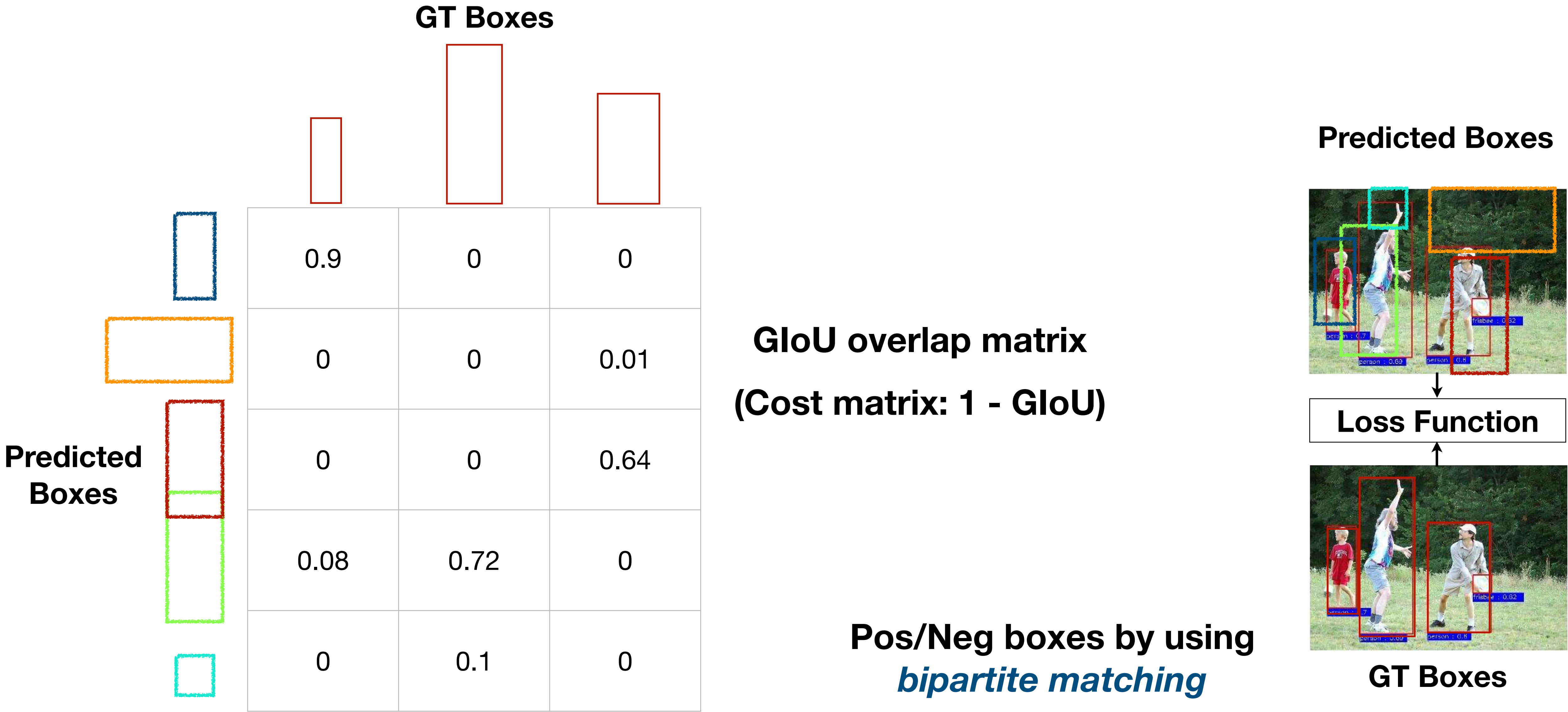


Loss Function

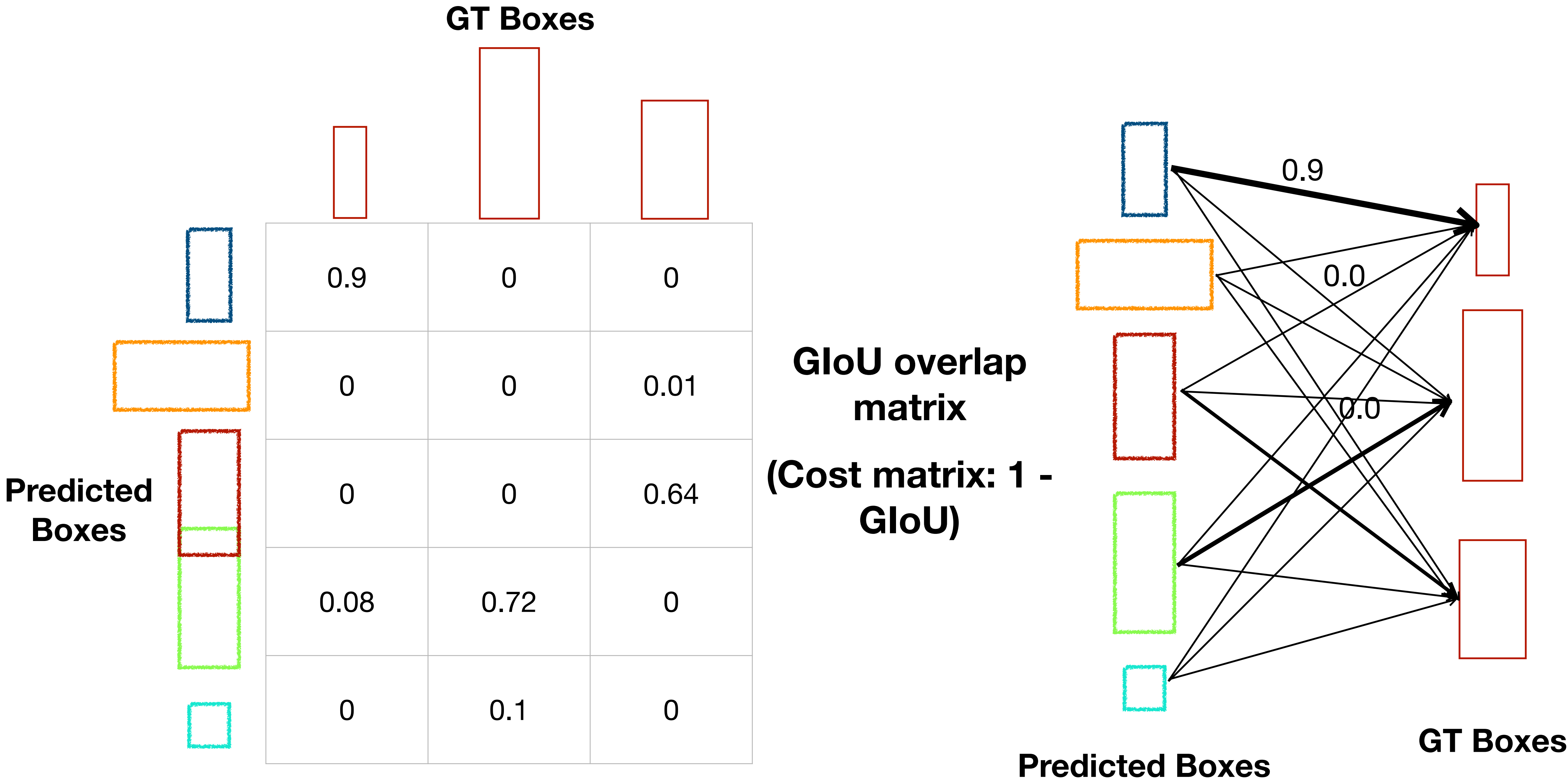


GT Boxes

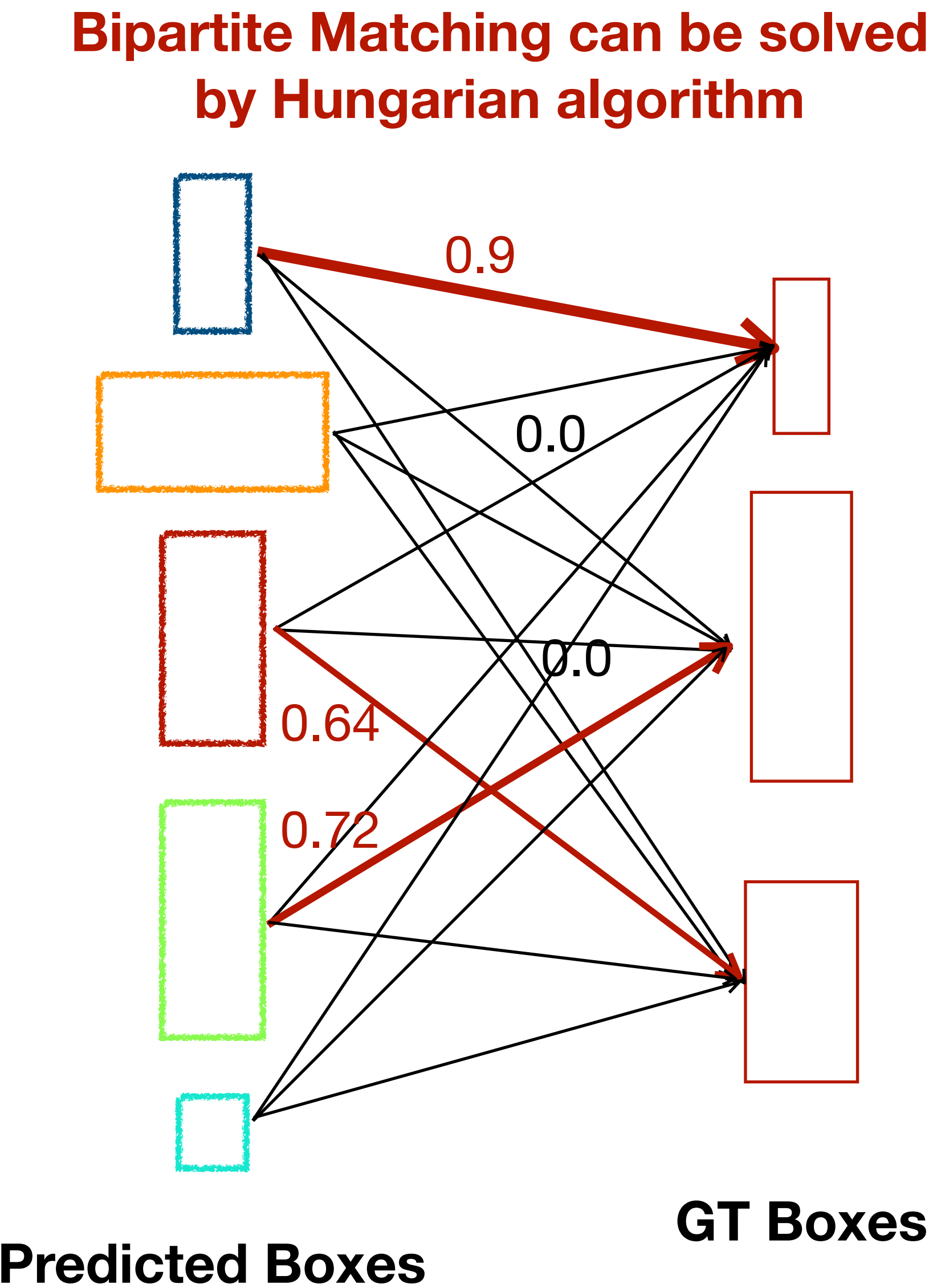
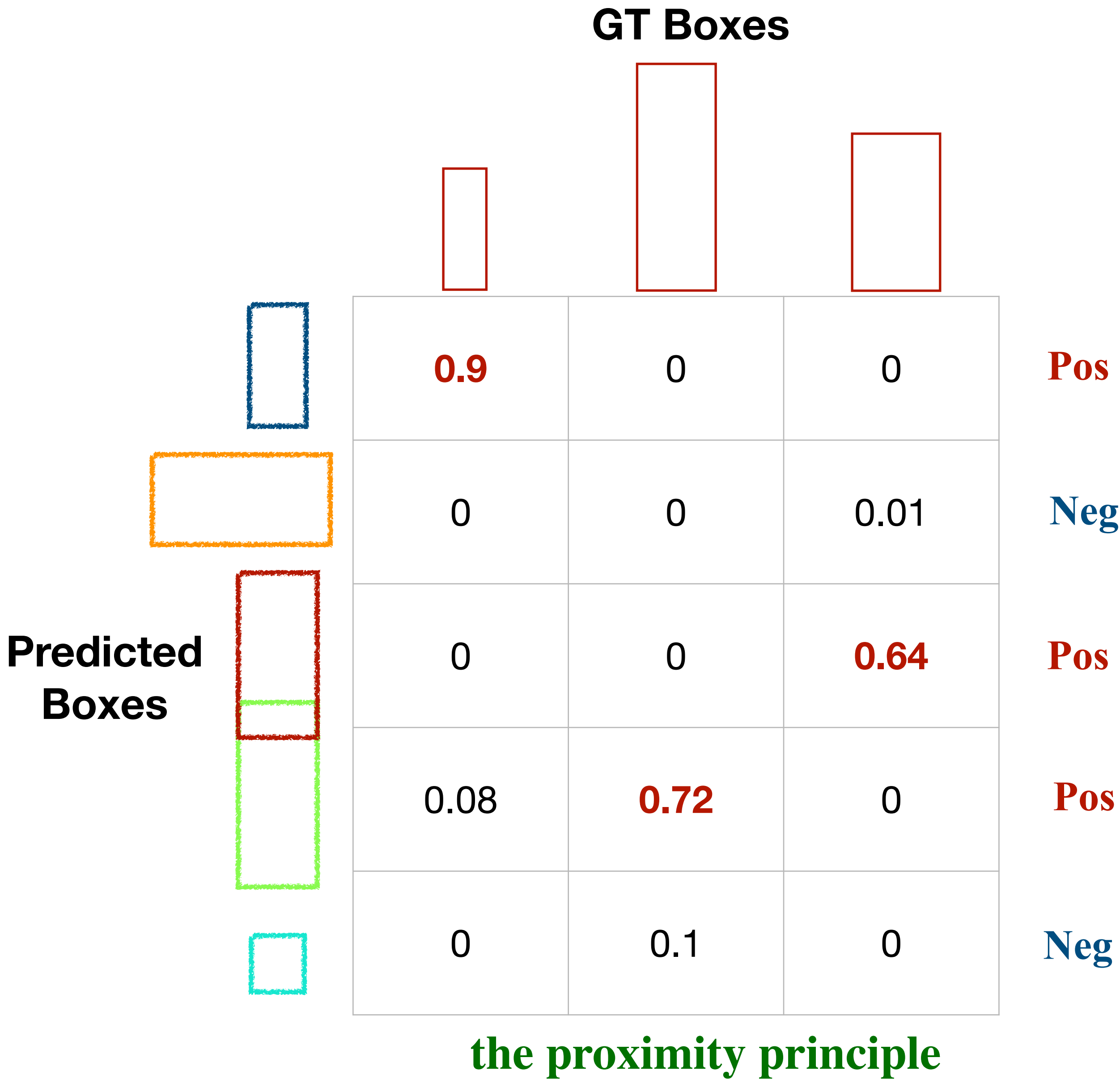
Object Detection with Transformers: DETR



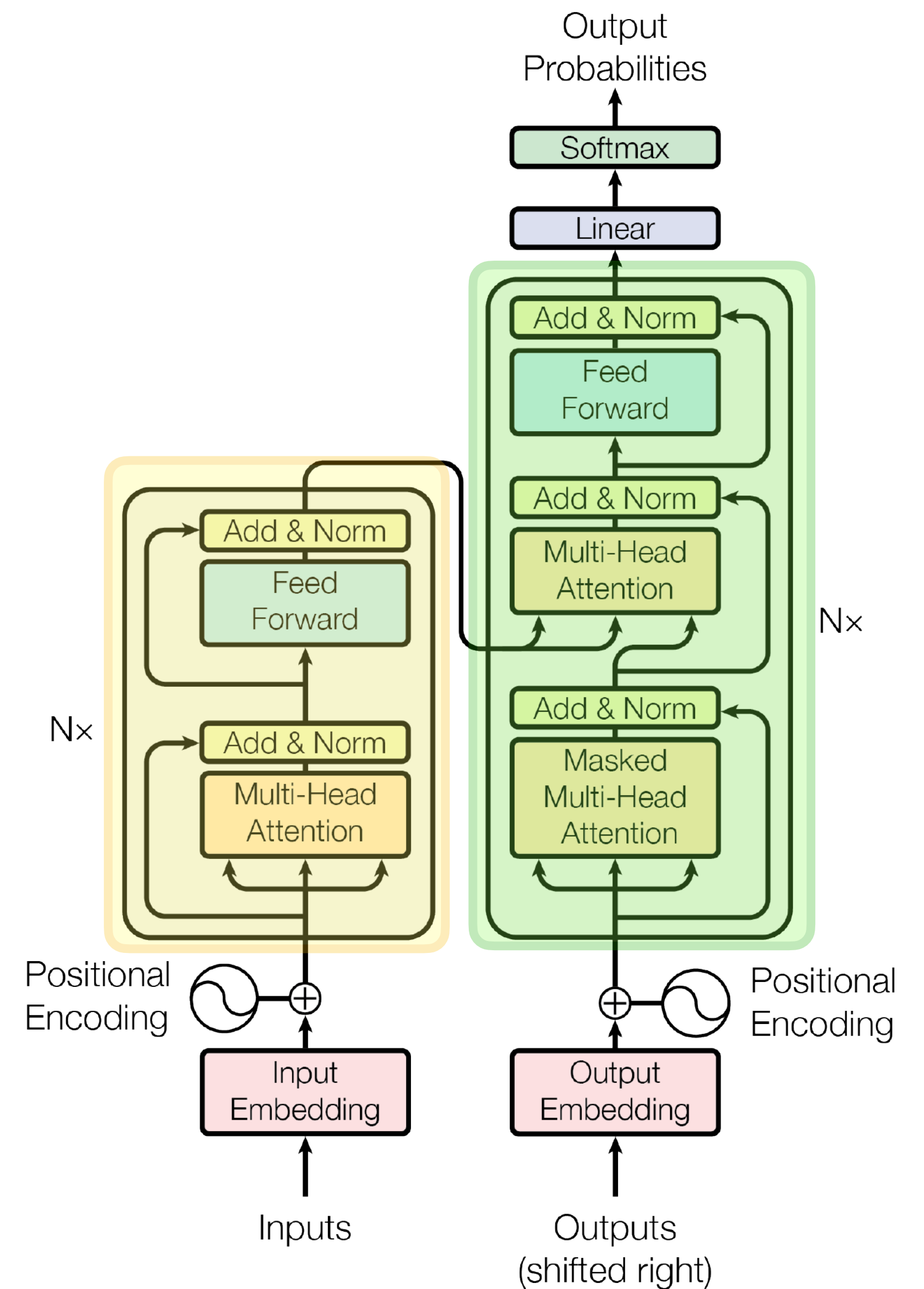
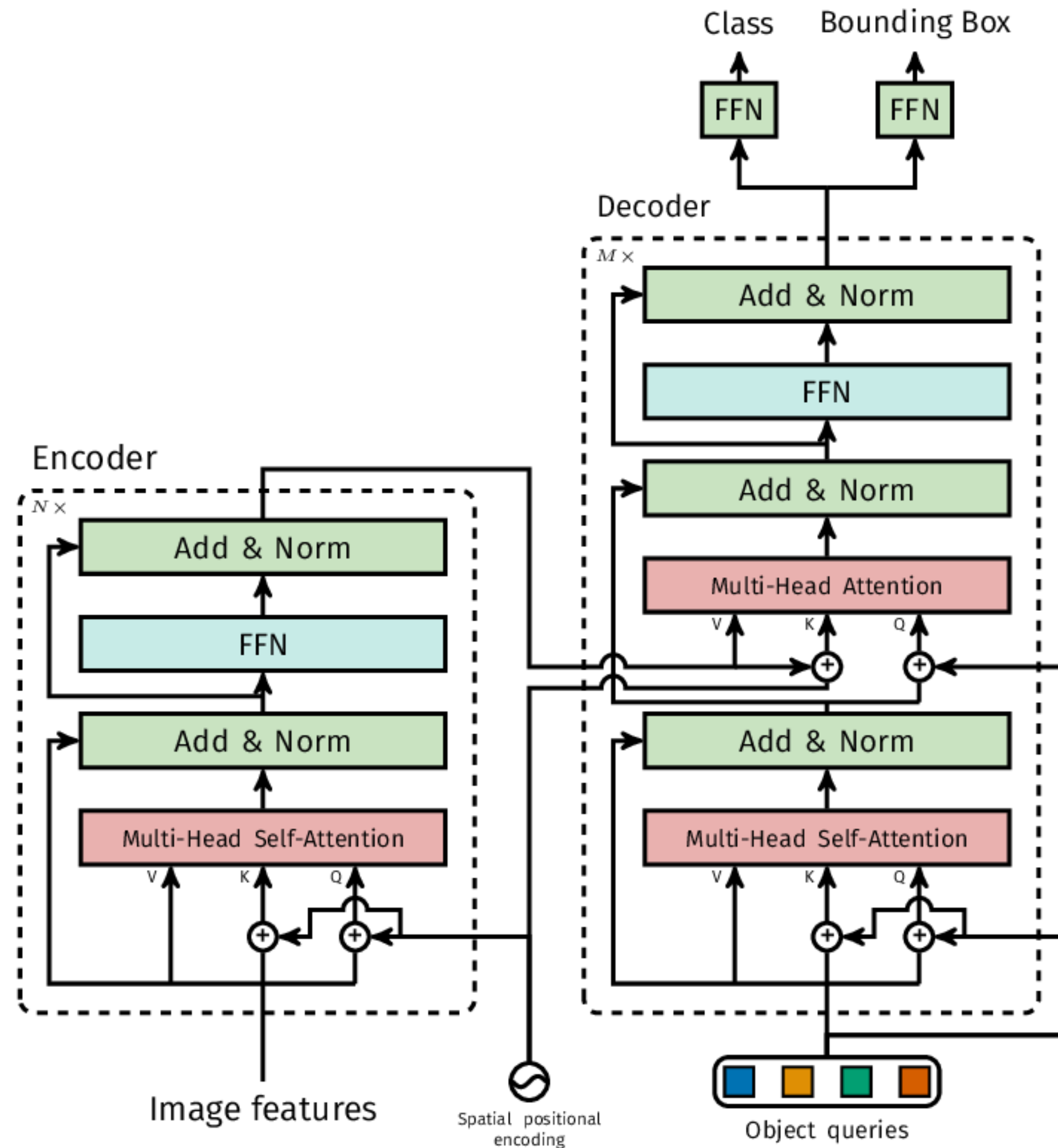
Object **D**etection with **T**ransformers: DETR



Object Detection with Transformers: DETR



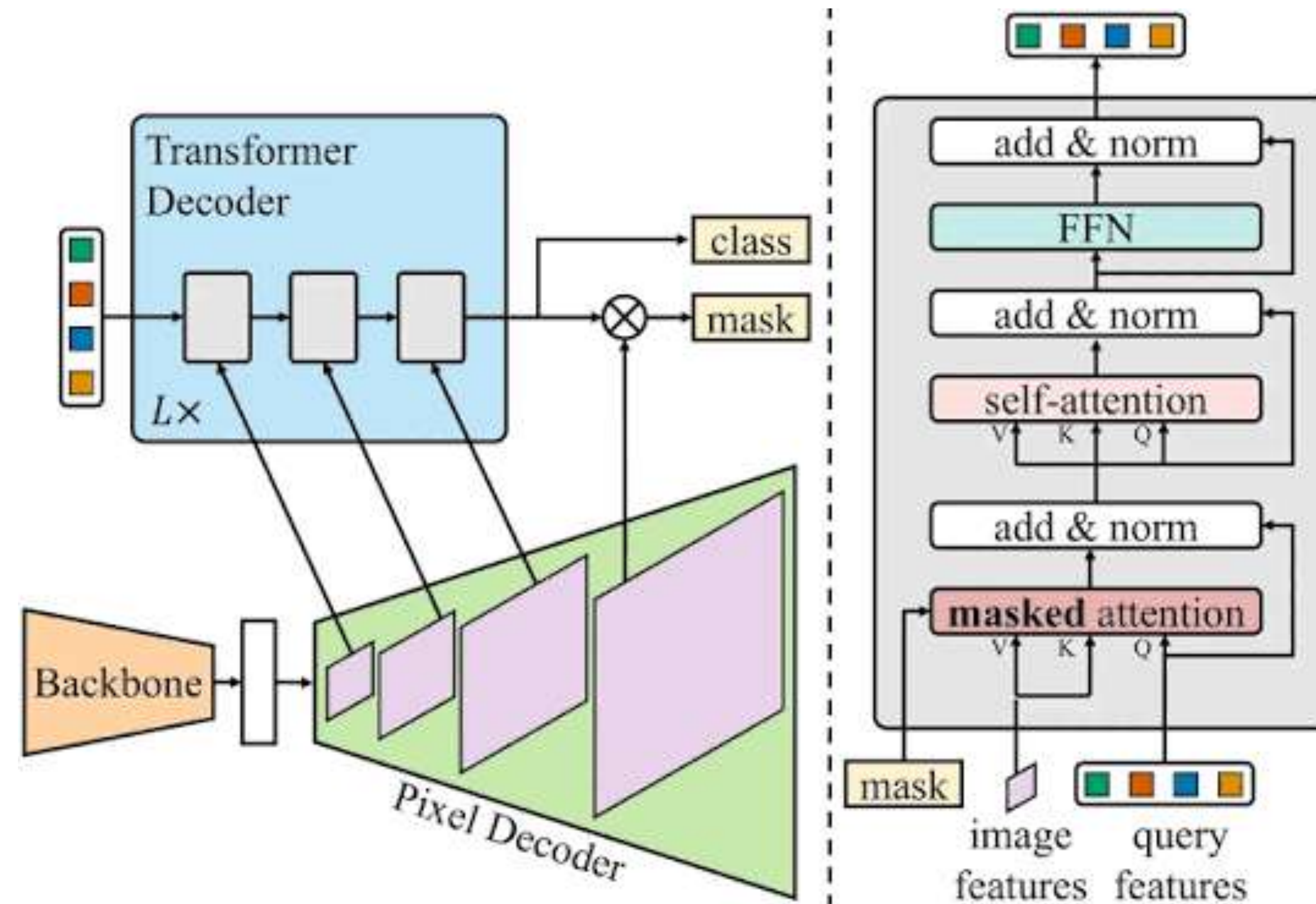
Object **D**etection with **T**ransformers: DETR



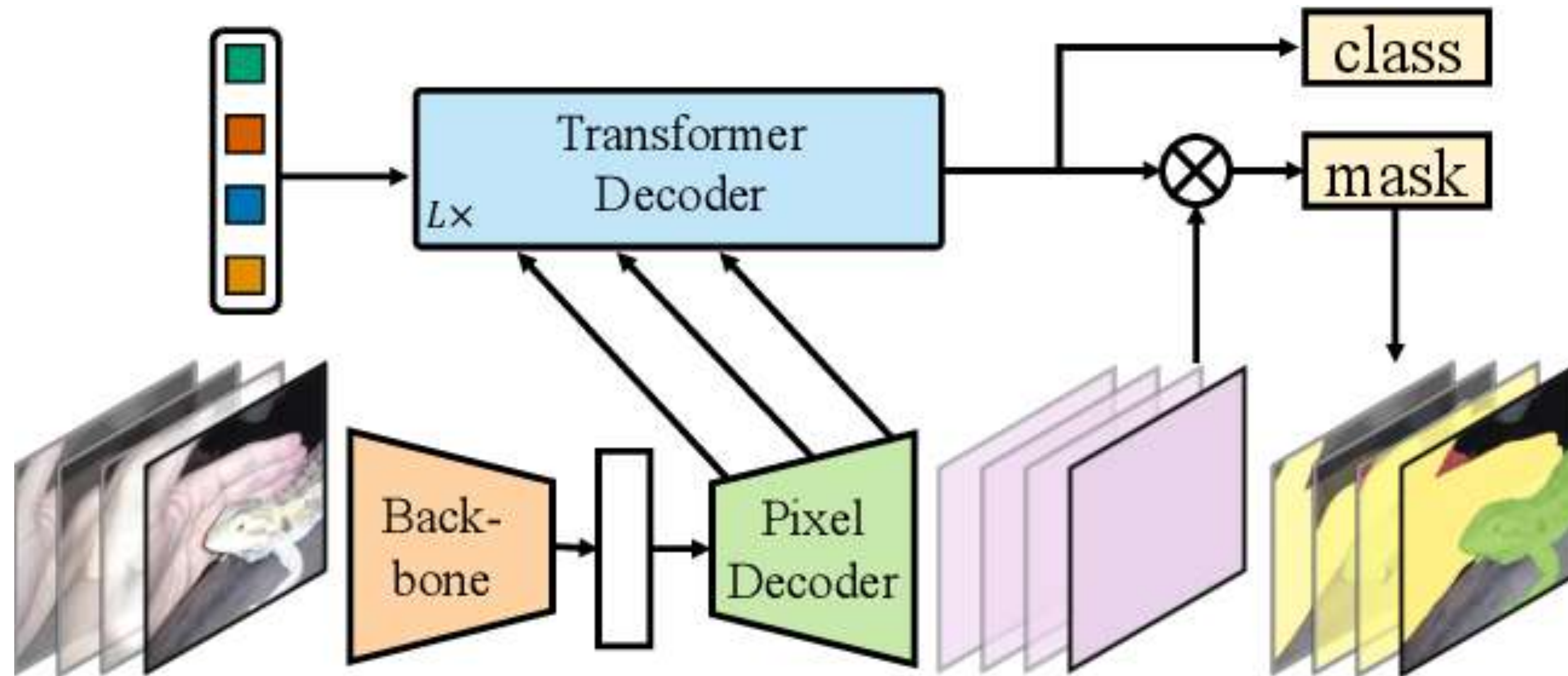
Transformer-based Instance Segmentation

DETR v.s. Mask2Former

Mask2Former: Masked-attention mask transformer for universal image segmentation



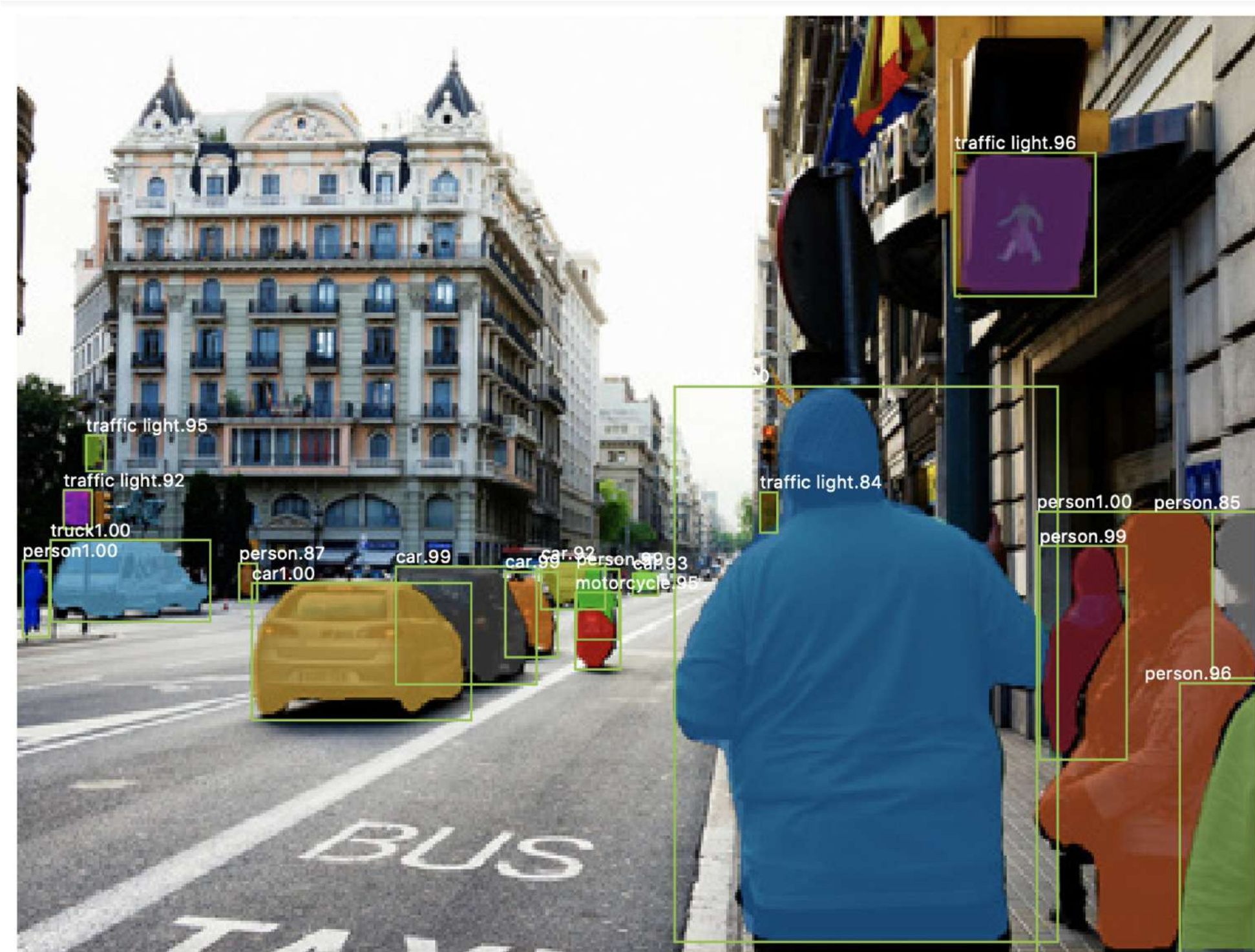
Mask2Former-Video



Q1: Why we use CNN as Backbone instead of vision transformer (ViT)?

Q2: How do we process the temporal information in videos? Next Tuesday!

Segmentation Visualization of YOLACT / Mask R-CNN / Mask2Former



Masking model trained on dataset of specific number of objects (80 in COCO)

Model outputs masks of all objects in that image that is one of the categories of interest

SAM: Segment Anything Models

What does it mean to have a segmentation foundation model?



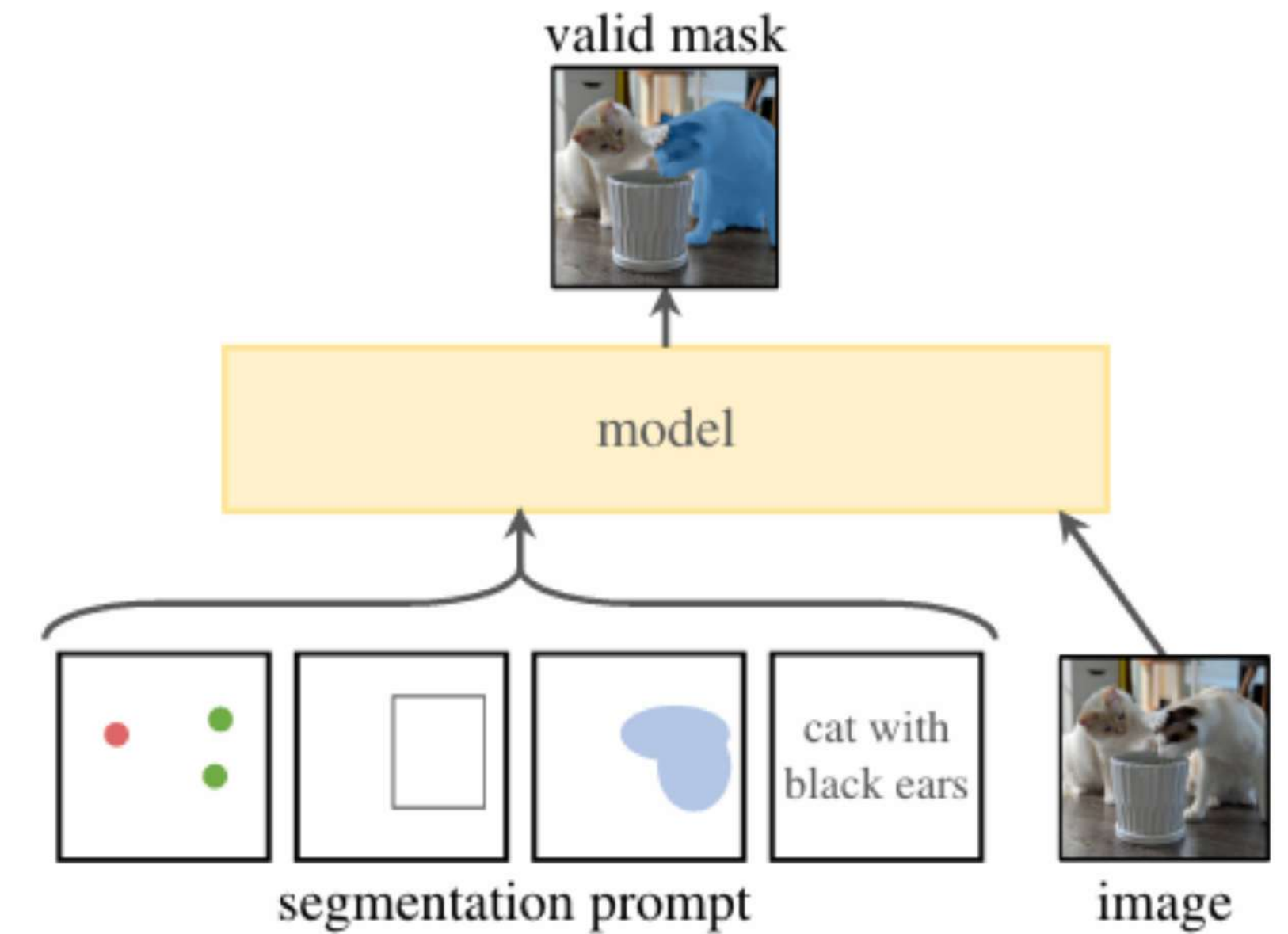
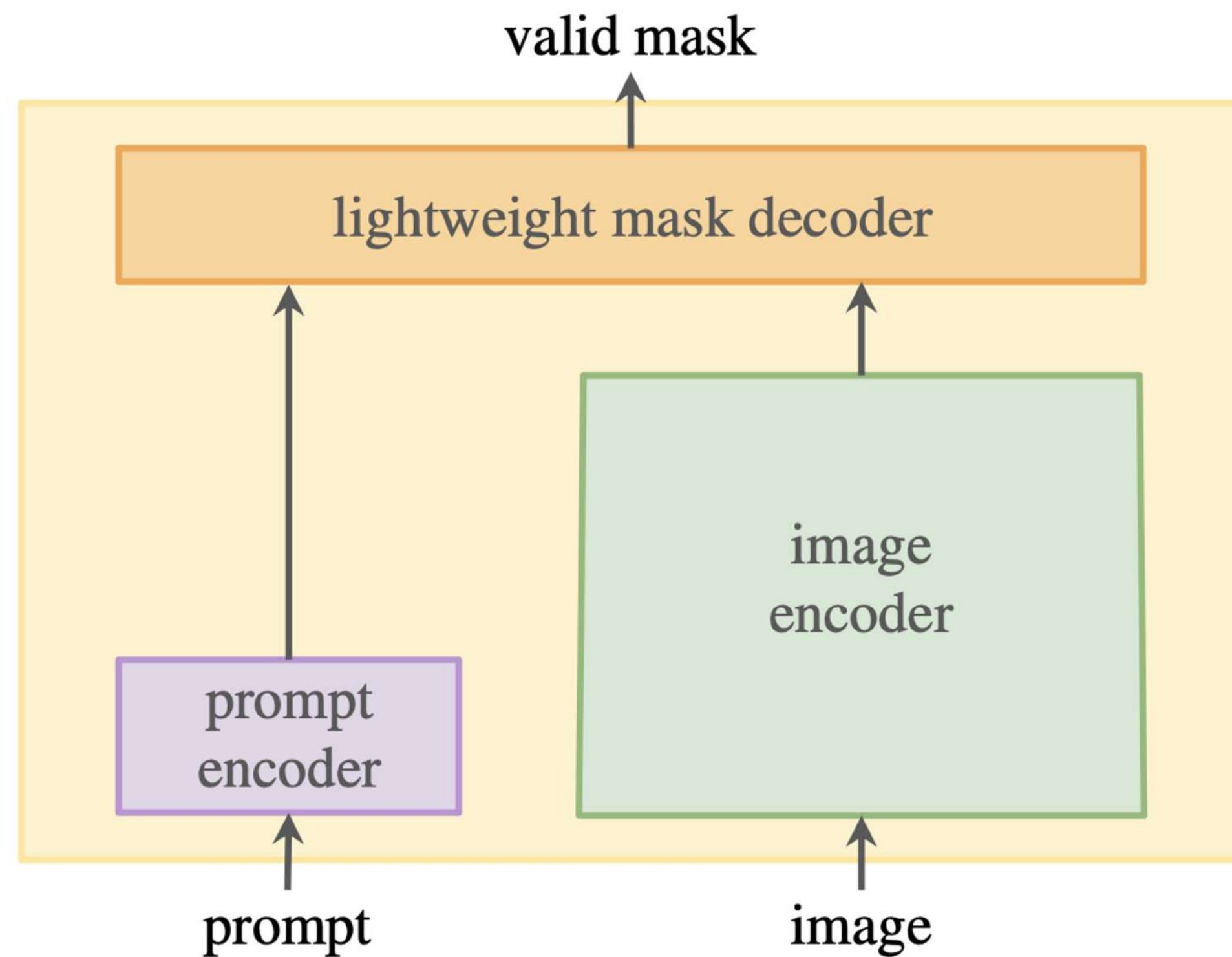
Masking model trained on a dataset of a huge number of categories *How to get the data?*

Model outputs mask of any objects that the user cares about *How to design a model supports segment anything?*

Images: Kirillov et al. Segment Anything. 2023.

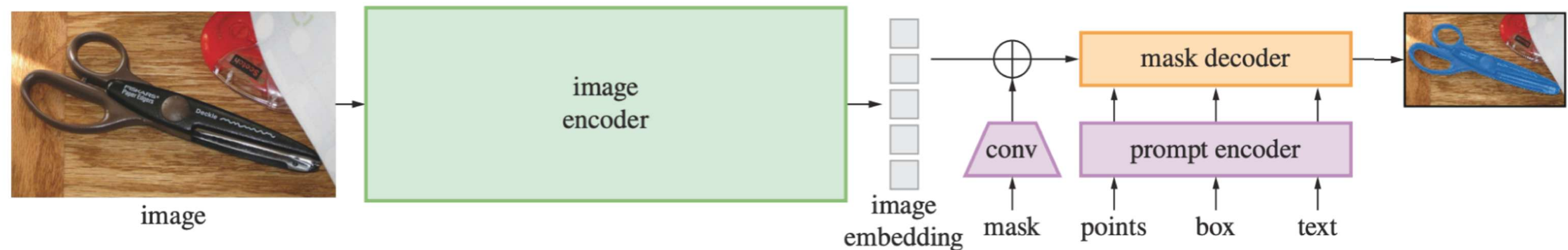
SAM: Segment Anything Models

Basic SAM Architecture



SAM: Segment Anything Models

SAM Architecture



How to add the correct Position Embedding?

How to handle the different numbers of coordinates in clicked points, boxes, masks?

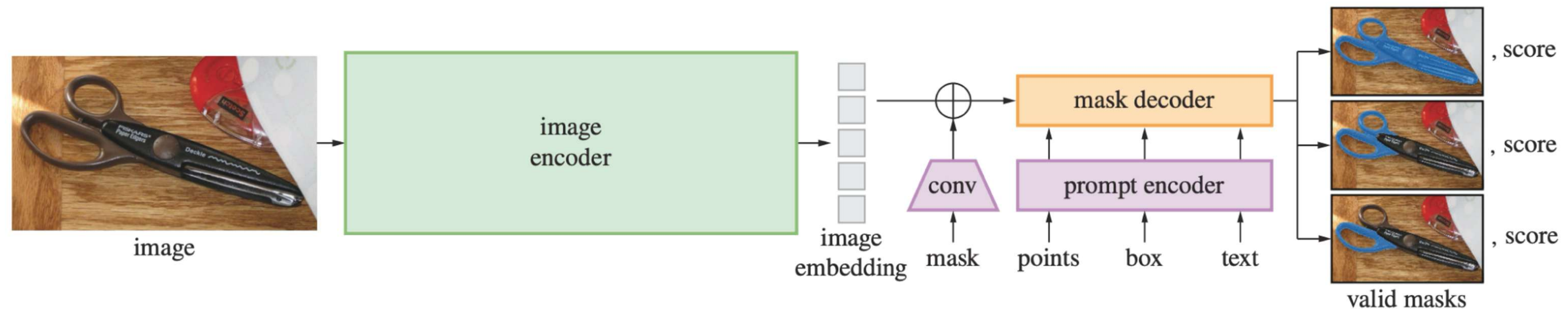
SAM: Segment Anything Models

Ambiguity in correct prompt



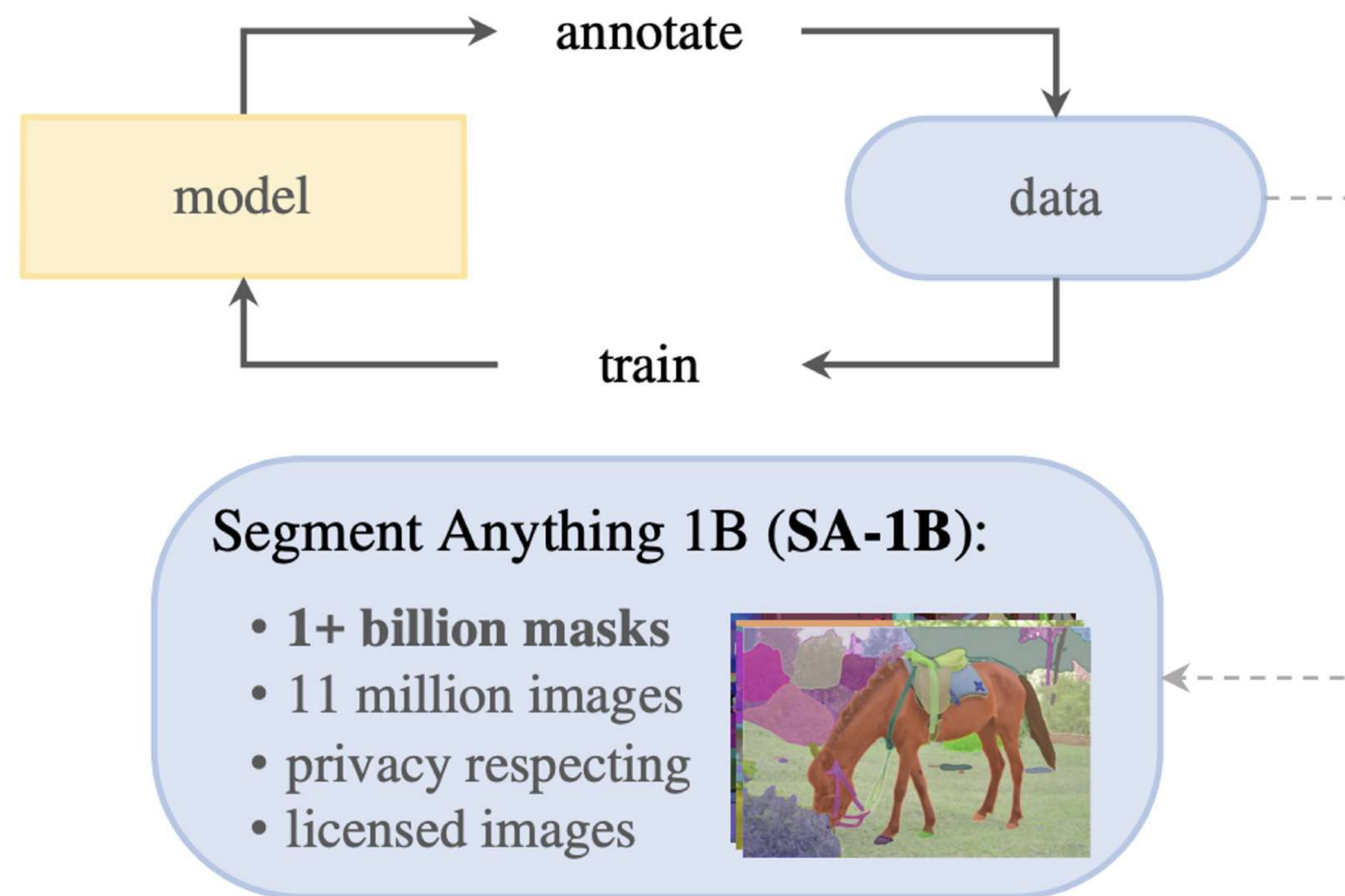
SAM: Segment Anything Models

Basic SAM Architecture



1. Loss only calculated with respect to best mask
2. Model also trained to output confidence score for each mask

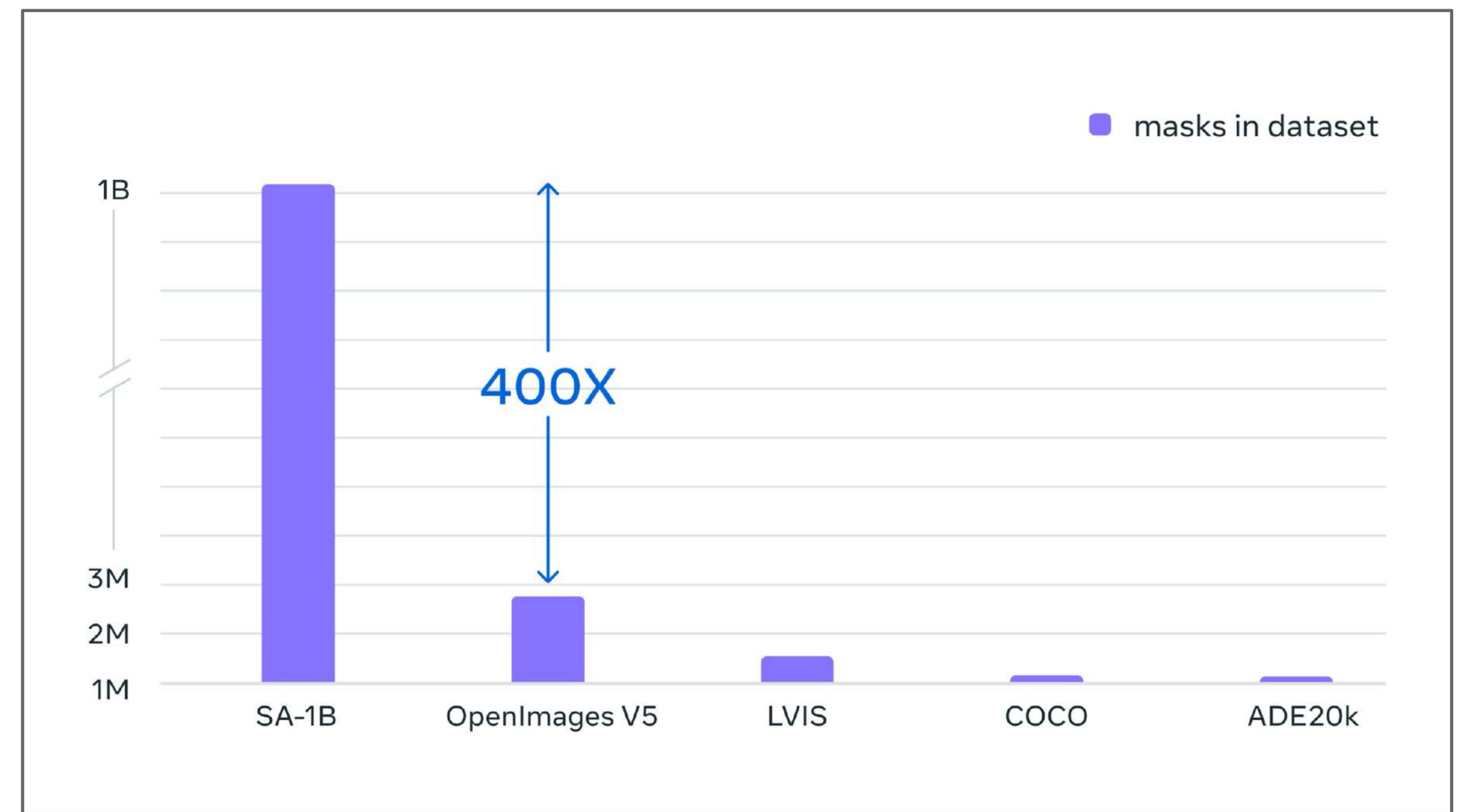
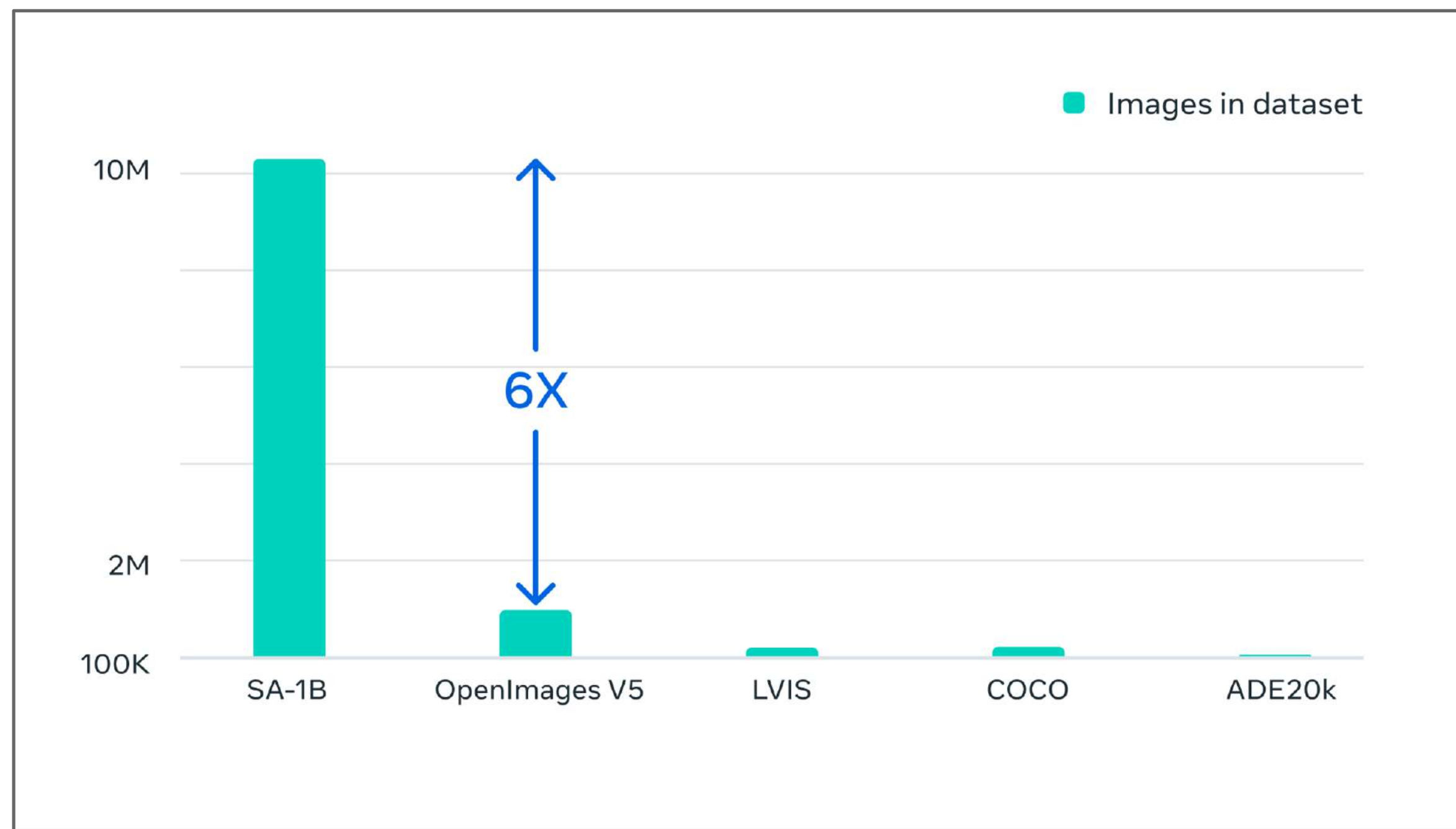
SA-1B: Dataset of Segment Anything Models



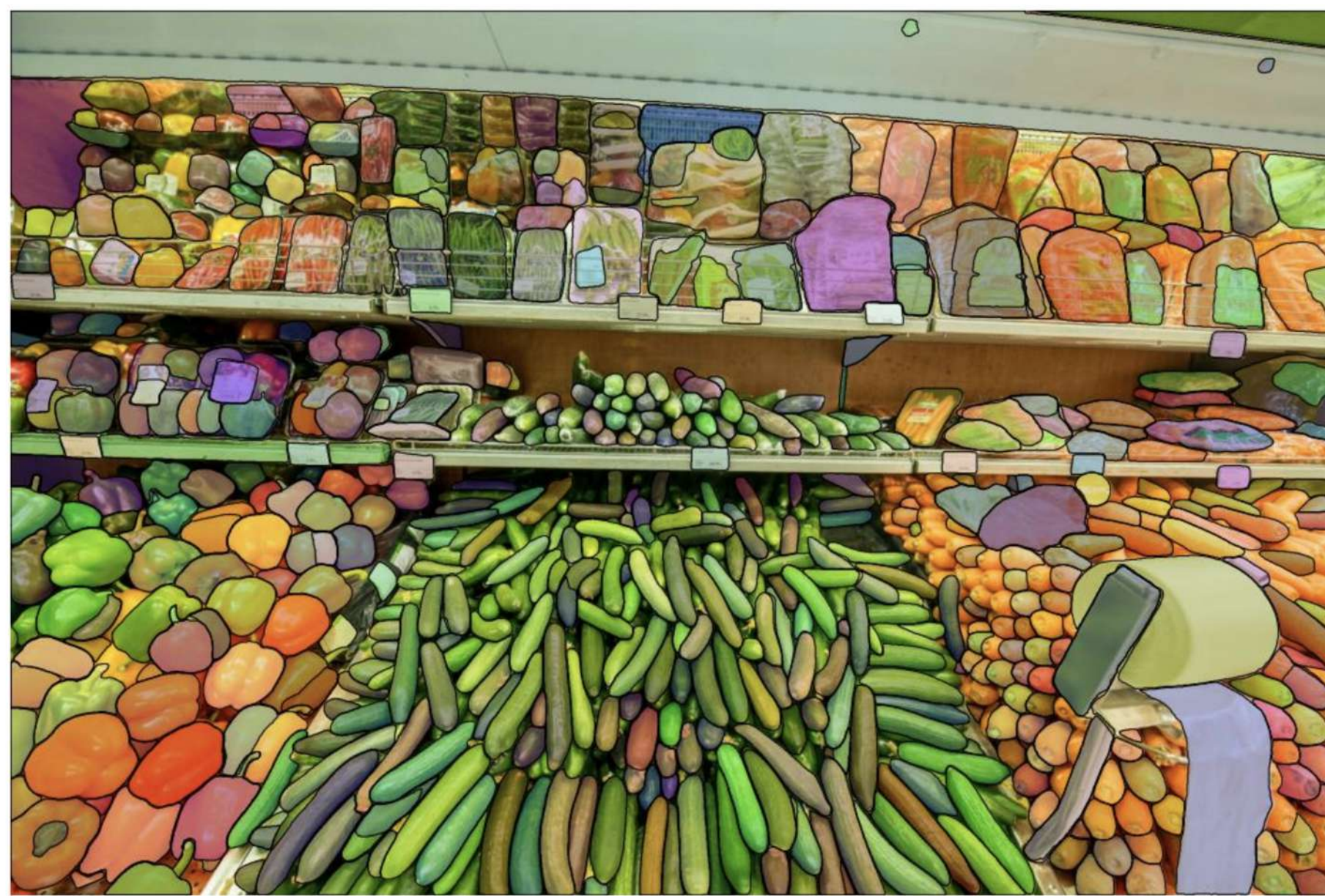
Data engine evolves through three stages:

- **Assisted-manual:** Annotators use clicks to generate masks with SAM and correct the results. SAM is repeatedly retrained on the collected annotations.
- **Semi-automatic:** SAM automatically labels some objects, while annotators label the remaining objects, increasing mask diversity.
- **Fully automatic:** SAM uses a regular grid of point prompts to generate masks, then filters low-quality and duplicate masks.

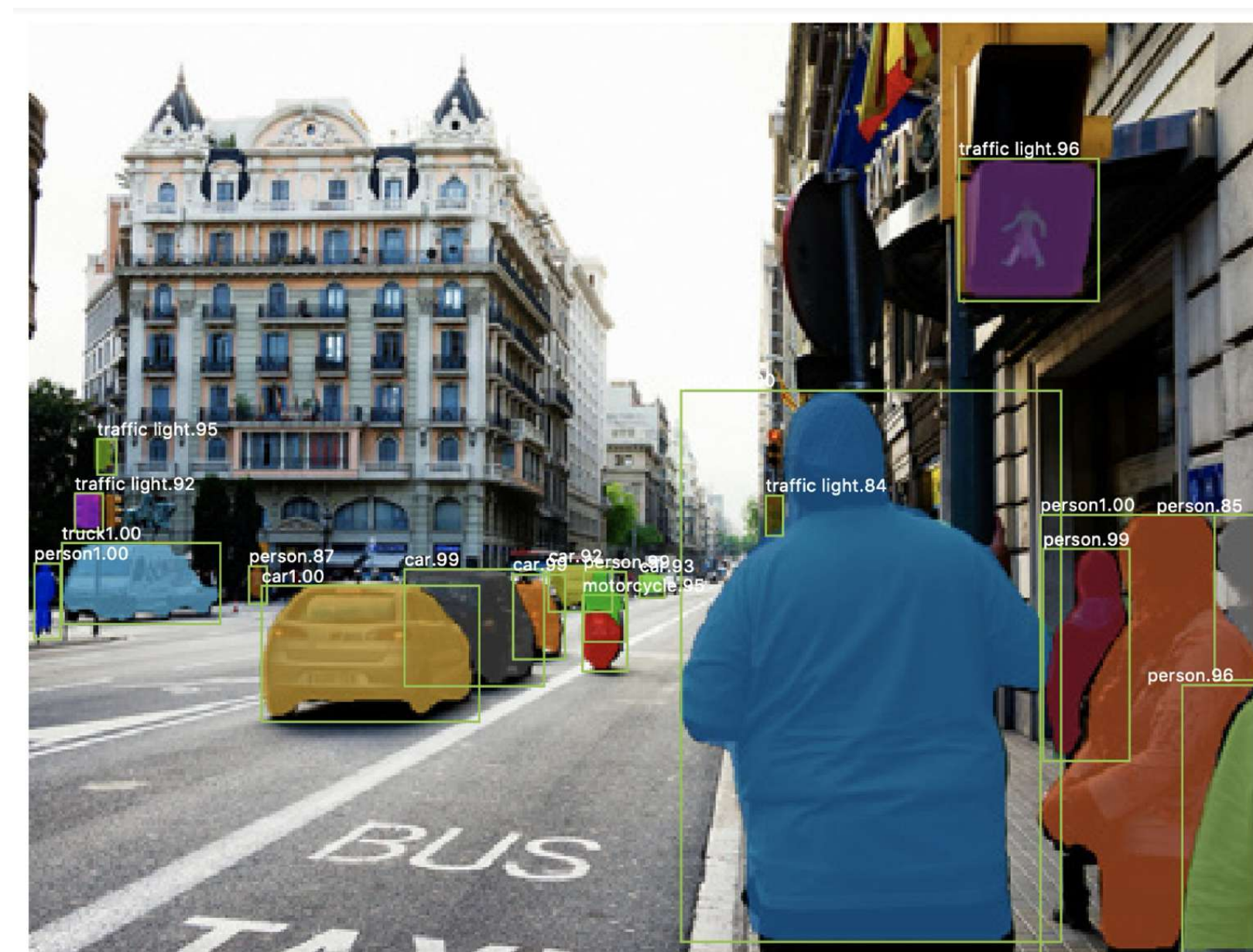
SA-1B: Dataset of Segment Anything Models



SAM: Segment Anything Models



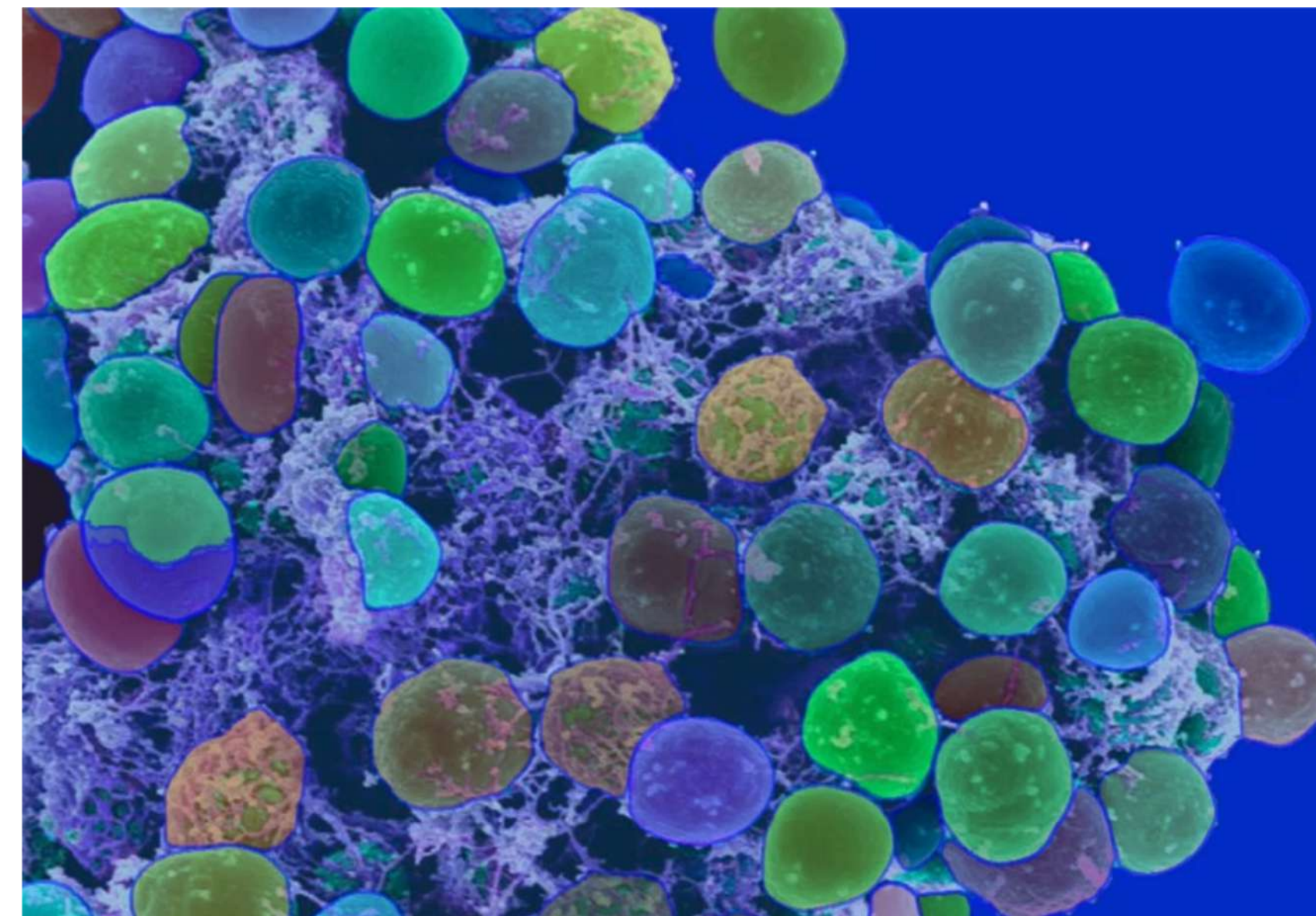
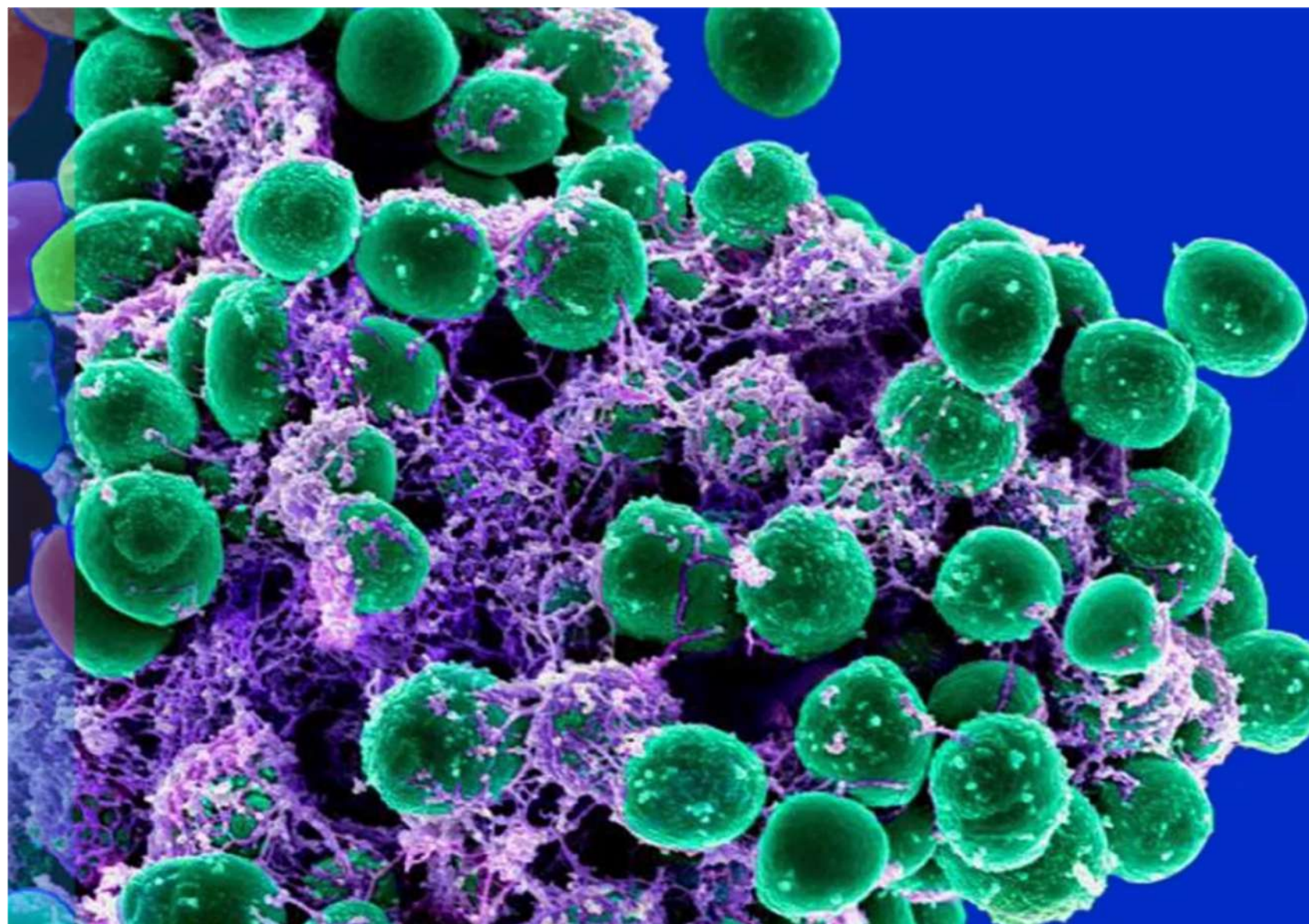
SAM Results



Mask R-CNN

SAM: Segment Anything Models

Zero-Shot with SAM



SAM: Segment Anything Models

Zero-Shot with SAM



Recap:

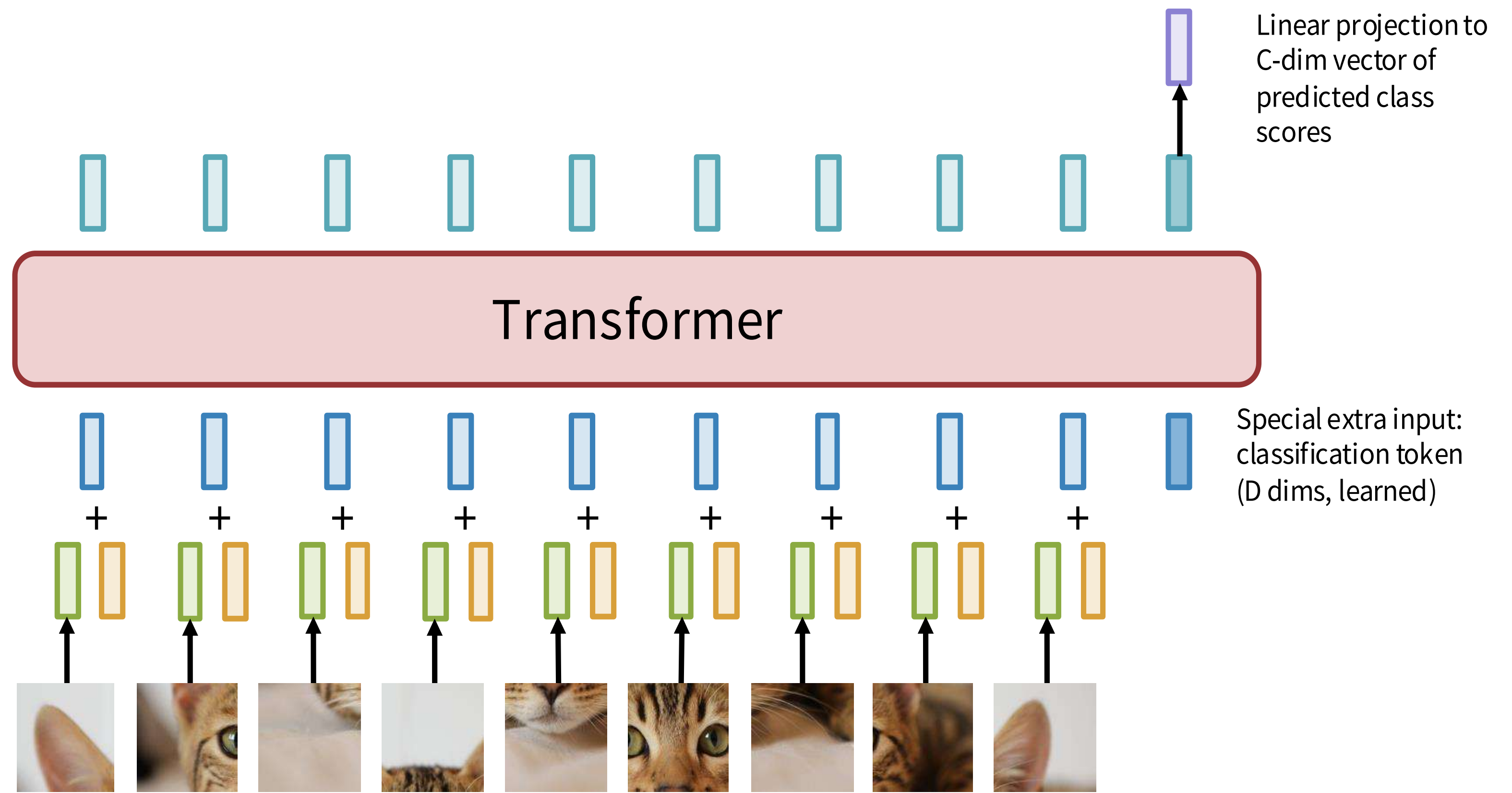
Vision Transformers (ViT)

- **Split non-overlapping patches**, so spatial continuity across patch boundaries is not explicitly preserved
- **Downsample 16x in the first step reduces spatial resolution too early**, making small objects harder to detect and fine boundaries harder to segment.

Add positional embedding:

Exact same as NLP Transformer!

Output vectors

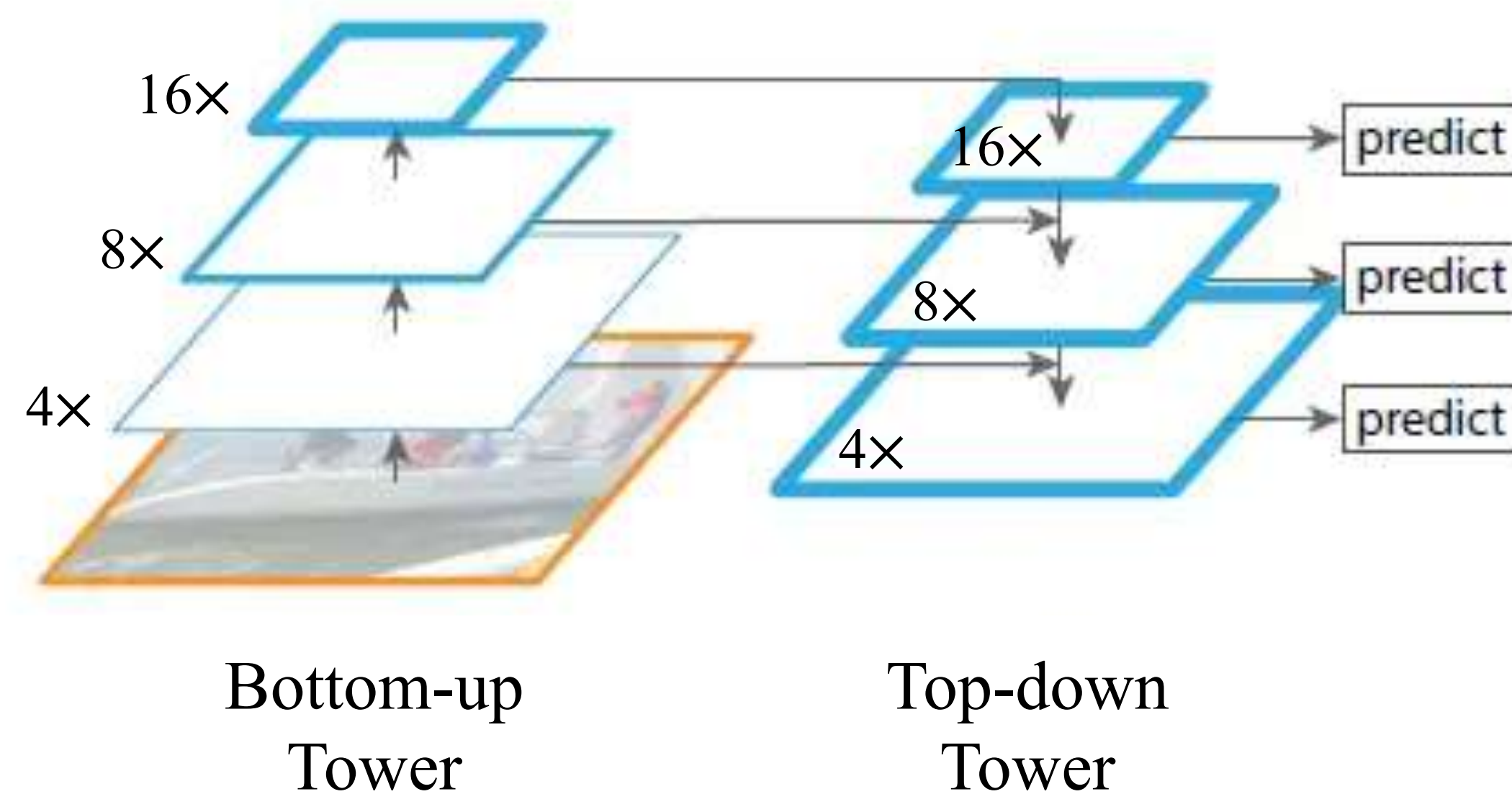
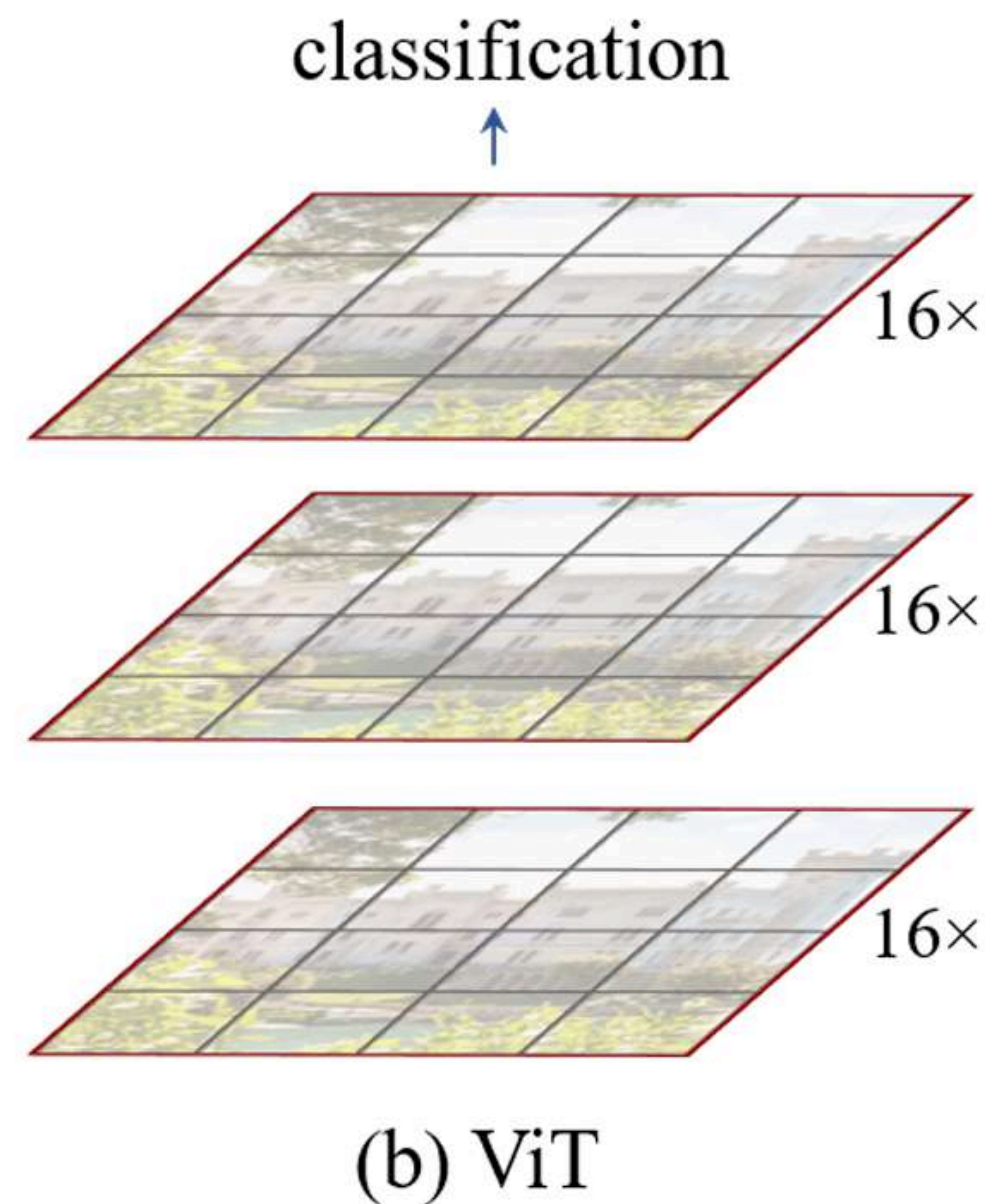


Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

[Cat image](#) is free for commercial use under a [Pixabay license](#)

Pyramid Feature Stages

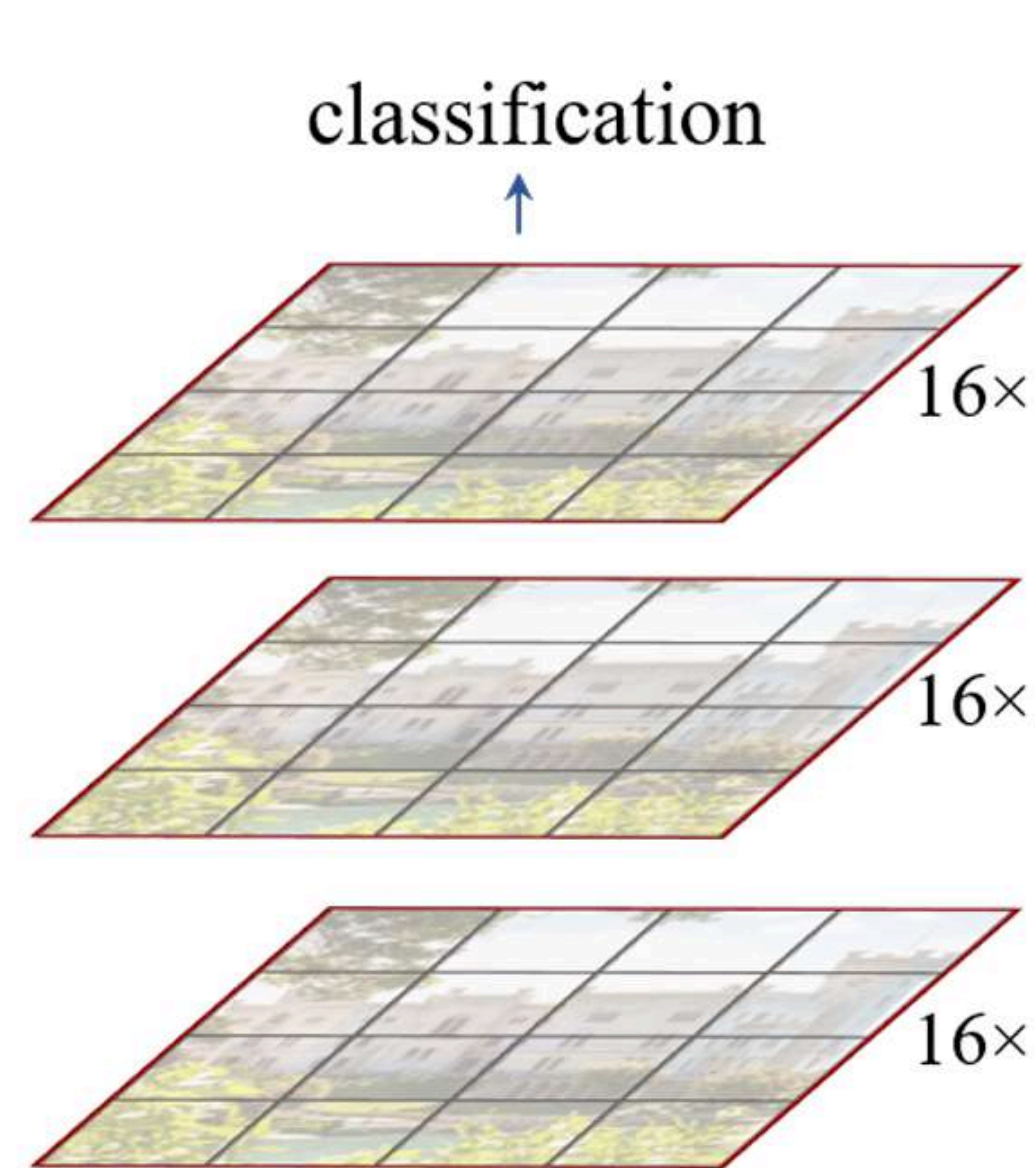
- **Downsample 16x in the first step**
reduces spatial resolution too early,
making small objects harder to detect and
fine boundaries harder to segment.



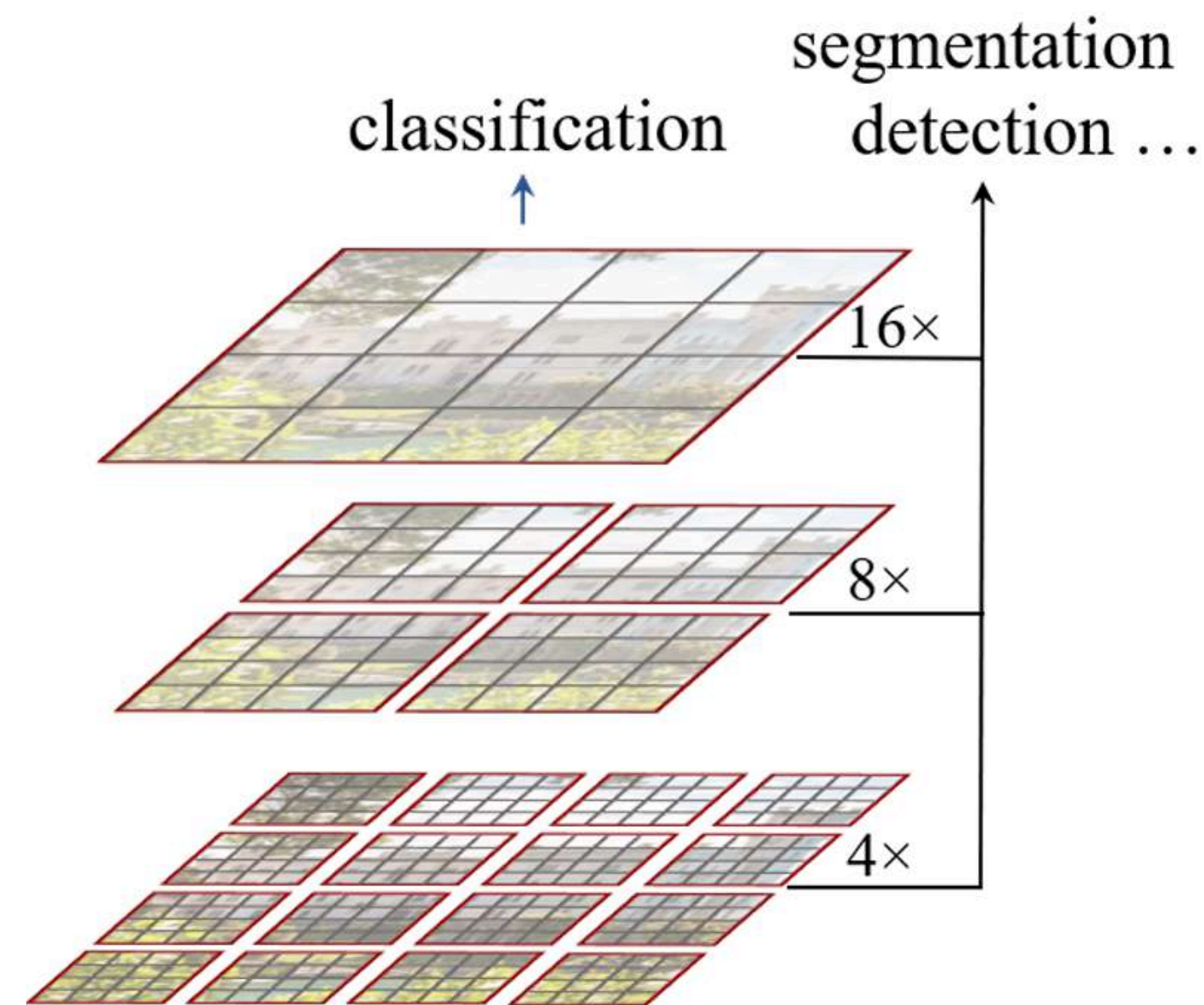
Swin Transformer: Hierarchical Vision Transformer using Shifted Windows

- **Downsample 16x in the first step**
reduces spatial resolution too early,
making small objects harder to detect and
fine boundaries harder to segment.

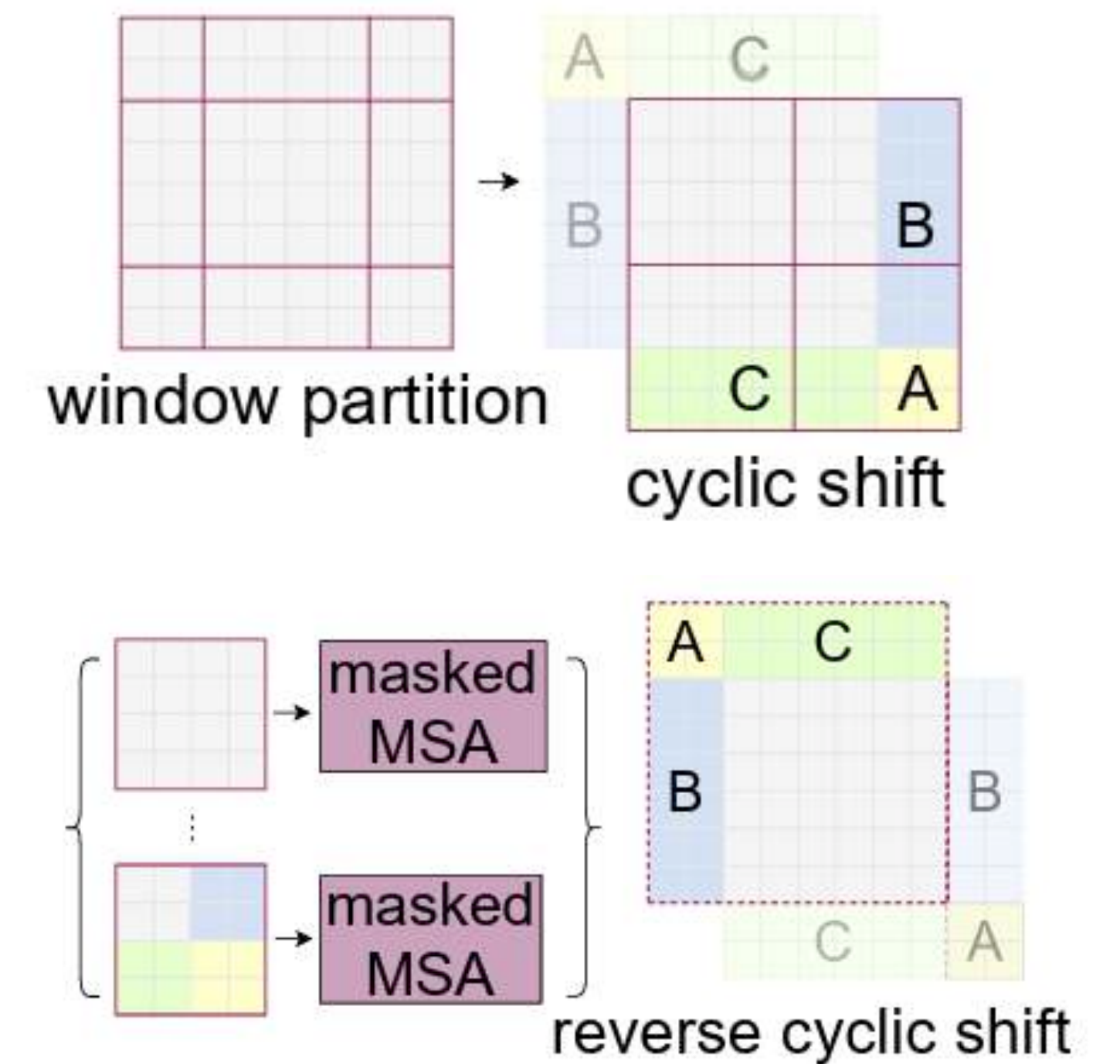
- **Split non-overlapping patches**, so
spatial continuity across patch
boundaries is not explicitly preserved



(b) ViT

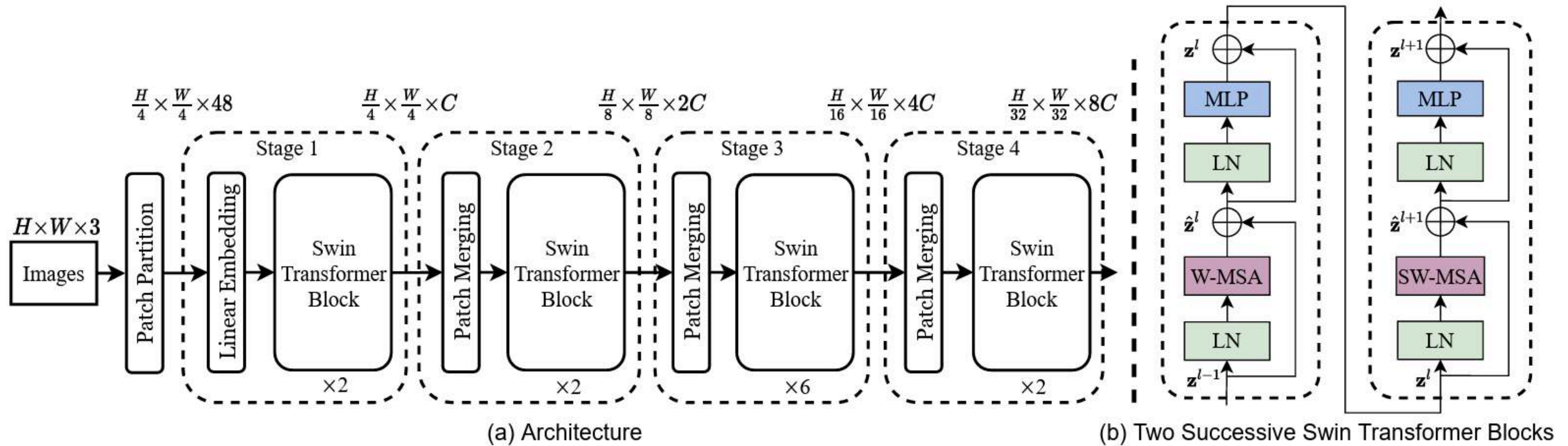


(a) Swin Transformer (ours)



Shifted Window based Self-Attention
(SW-MSA)

Swin Transformer: Hierarchical Vision Transformer using Shifted Windows



Next Lecture: Motion & Video Understanding

Video: $T \times 3 \times H \times W$



3D Convolution: High Level Idea

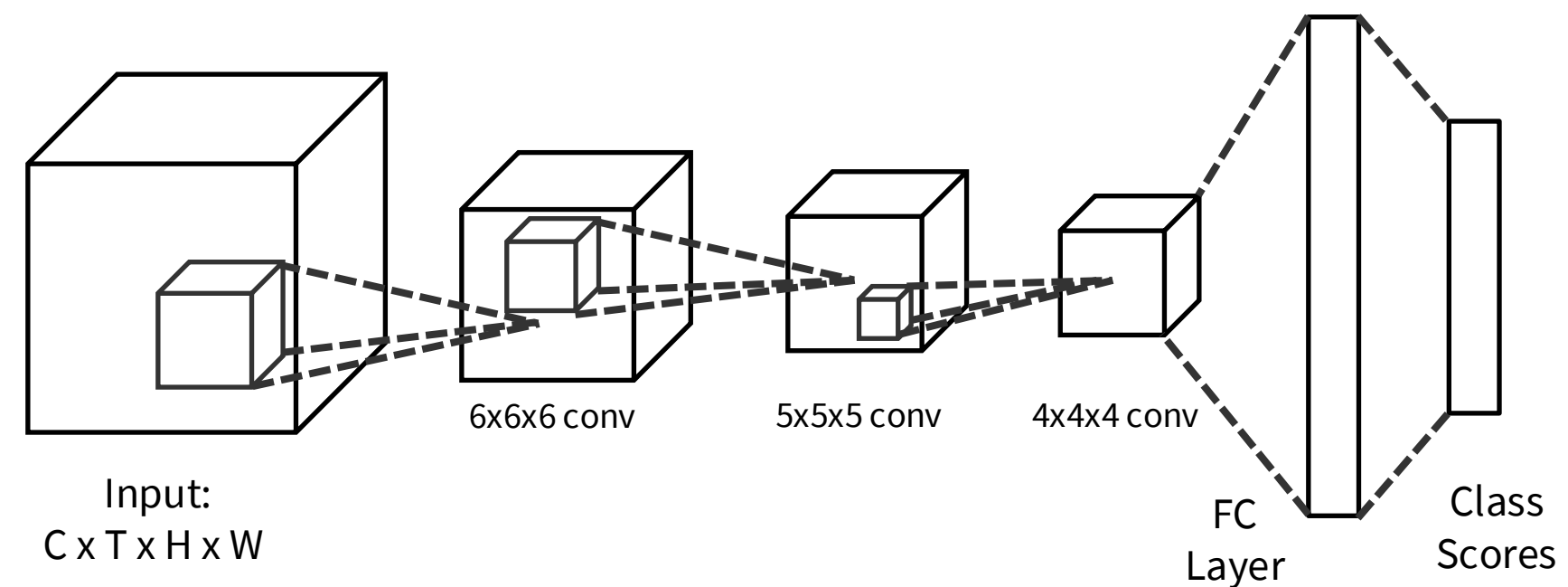
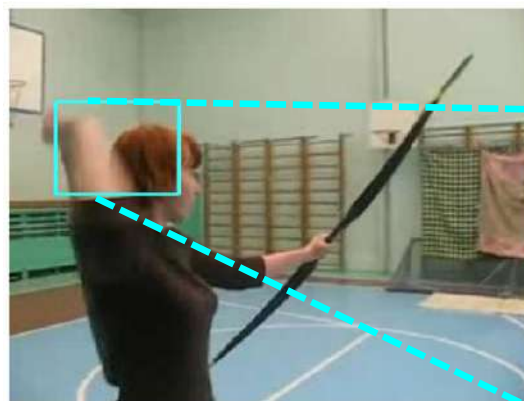


Image at frame t



Optical flow gives a displacement field F between images I_t and I_{t+1}

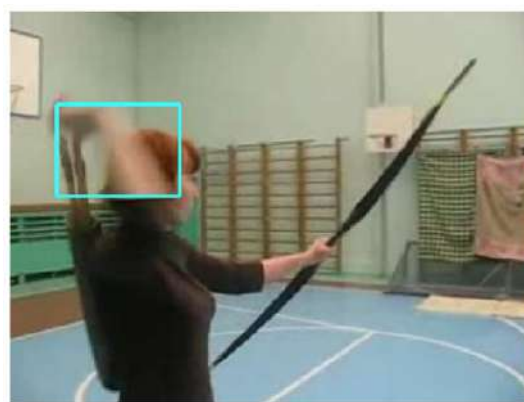
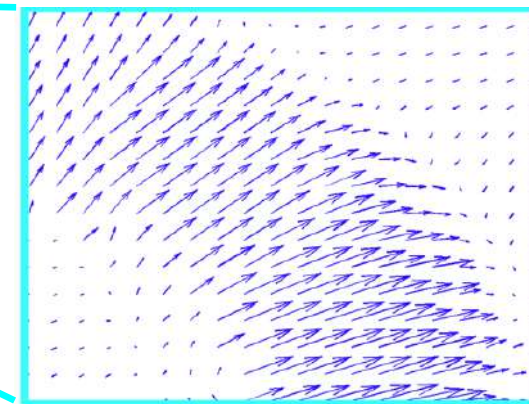


Image at frame $t+1$

Tells where each pixel will move in the next frame:
 $F(x, y) = (dx, dy)$
 $I_{t+1}(x+dx, y+dy) = I_t(x, y)$

Video Swin Transformer (ViViT)

